Mini Claude Code

Python s01 -> s10 逐层复刻完整工程教程

Agent Loop / Tools / Permissions / Context / Memory / Skills / Hooks / Subagents / Sandbox / Multi-Agent

核心方法

每一层只增加一个主要抽象：sN = sN-1 + 一个可解释能力。重点同时回答 know-why（为什么需要这一层）与 know-how（代码如何落地、如何验证、哪里容易错）。

版本：2026-08-13 | Python 3.11+ | 教学实现

Mini Claude Code s01-s10 | 1

0. 如何使用这本教程

这不是“照着 Claude Code UI 抄功能”的教程，而是一条可执行的 Agent Runtime 演化路径。你可以从 s01 直接运行，然后每完成一章就切换到下一目录，观察代码为何需要新增一个层。所有代码均为教学用原创实现。

0.1 成功标准

- 理解最小 tool-use 状态机，知道 Host 为什么是 Agent 的真实执行边界。
- 能把工具能力、权限策略、上下文治理和持久知识拆成互不混淆的层。
- 能实现 progressive disclosure Skill、deterministic Hook 和 context-isolated Subagent。
- 能解释“Permission != Sandbox”“Subagent != Agent Team”“Memory != Context”。
- 最终能运行一个支持 Git worktree 并行 Worker 的 Mini Multi-Agent Runtime。

0.2 安全与准确性边界

> **重要**
> s09 的 soft sandbox 只提供 cwd、环境变量净化和 timeout，不是强 OS 安全边界。Bubblewrap 例子也只是教学基线。任何真实生产 Coding Agent 都应再加容器/VM、网络代理、密钥隔离、审计、资源限额和组织策略。

另外，本书把公开文档中的 Claude Code 机制映射为一个可学习的 Python Runtime，但不声称代码结构、函数名、目录或内部策略与 Anthropic 私有实现相同。

0.3 全局演化图

```
User Prompt
    |
    v
s01 Agent Loop
    | + structured tools
    v
s02 Tool Runtime
    | + policy gates
    v
s03 Permissions
    | + budget / compact
    v
s04 Context Manager
    | + persistent knowledge
    v
s05 Memory
    | + progressive procedures
    v
s06 Skills
    | + deterministic lifecycle
    v
s07 Hooks
    | + isolated contexts
    v
s08 Subagents
    | + execution containment
    v
s09 Sandbox
```

```
    | + worktrees / parallel workers / mailbox
    v
s10 Multi-Agent Runtime
```

0.4 十层总览

Stage	新增抽象	核心问题	最终收益
s01	Agent Loop	模型如何连续调用工具?	最小闭环
s02	Tool Runtime	如何让能力可扩展、可约束?	结构化工具 + dispatch
s03	Permission	谁决定副作用能否执行?	Host policy
s04	Context	历史越来越长怎么办?	统计 + compact
s05	Memory	哪些信息要跨会话?	规则 + 持久学习
s06	Skills	重复 SOP 如何按需加载?	progressive disclosure
s07	Hooks	必须发生的动作如何保证?	deterministic lifecycle
s08	Subagents	探索如何不污染主 Context?	context isolation
s09	Sandbox	批准后如何限制 blast radius?	execution containment
s10	Multi-Agent	并行修改如何避免互相踩文件?	worktree + workers

Mini Claude Code s01-s10 | 3

1. 环境、运行方式与学习约定

1.1 依赖

- Python 3.11 或更高版本。
- `anthropic`：Messages API。
- `python-dotenv`：本地开发加载环境变量。
- `pyyaml`：s06 以后解析 Skill/Subagent frontmatter。
- Git：s10 worktree 必需；s01-s09 不强制。

1.2 推荐初始化

```
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install anthropic python-dotenv pyyaml
export ANTHROPIC_API_KEY=...
export ANTHROPIC_MODEL=<your-model-id>
cd s01_agent_loop
python code.py
```

为什么不在代码里写死 model ID?

模型命名和账号可用范围会变化。教程通过 ANTHROPIC_MODEL 注入具体模型，使 Runtime 的架构与某个时间点的模型名称解耦。

1.3 API 协议必须先理解

Claude Platform 当前公开文档把客户端工具循环明确描述为：发送 messages + tools；模型返回 stop_reason=tool_use 与一个或多个 tool_use block；Host 执行；再发送包含匹配 tool_use_id 的 tool_result；重复直到模型停止请求客户端工具。[R1][R2]

```
messages + tools
      |
      v
Claude response
      | stop_reason == tool_use
      v
scan tool_use blocks
      |
      v
Host executes tools
      |
      v
user content: tool_result(tool_use_id=...)
      |
      +----------------------> next Messages API call

otherwise -> final answer / other stop reason
```

Mini Claude Code s01-s10 | 4

2. s01 - Agent Loop

从 0 到最小可运行 Agent Loop

观察项	本阶段
阶段目录	s01_agent_loop
文件数量	2
新增文件	README.md, code.py
继承/修改文件	无
Python 行数	150

KNOW-WHY：为什么这一层必须存在

- Agent 的最小闭环不是“模型会调用函数”，而是 Host 必须把 tool_use 结果执行后，以 tool_result 再喂回模型。只要这个闭环正确，后续所有能力都可以在它外面分层叠加。
- 把 stop_reason 当作协议状态机，而不是把一次 API 调用当成一次完整任务。模型可以经历多轮工具调用后才给出最终自然语言答案。
- 工具副作用由 Host 执行，这意味着真正的安全边界、审计、权限和超时必须在 Host 侧实现，而不能只依赖模型提示词。

KNOW-HOW：这一层具体怎么长出来

1. 只暴露一个 bash 工具，先让协议足够简单。
2. 每次模型响应都原样加入 assistant history；如果 stop_reason == tool_use，则扫描所有 tool_use block。
3. Host 执行命令，构造带相同 tool_use_id 的 tool_result，并以 user content block 形式加入 history。
4. 循环直到 stop_reason 不再是 tool_use。

数据流 / 控制流

```
User -> Messages API -> tool_use -> Host/bash -> tool_result -> Messages API -> ... -> final
```

最容易踩的坑

- 只把工具结果发回模型，却忘了把上一条 assistant tool_use 响应也加入 history。
- 假设 response.content[0] 一定是 tool_use；实际上文本块和多个 tool_use 可以混合出现。
- 把模型输出直接当 shell 命令执行，却没有权限、超时或隔离。这是 s01 故意留下、在后续阶段逐步补齐的缺口。

本阶段验收清单

- 让 Agent 回答当前工作目录，并观察是否经历一次 bash -> tool_result -> end_turn。
- 让 Agent“检查目录后总结项目”，观察是否能连续调用多次工具。
- 制造一个会返回非 0 exit code 的普通命令，确认错误仍作为 tool_result 返回模型而不是让 Host 崩溃。

协议不变量

- 每个 client tool_use 的结果必须通过相同 tool_use_id 对应回去。
- assistant 的 tool_use 响应属于历史的一部分，必须保留。
- 一个响应可能有文本 + 多个 tool_use，因此处理逻辑必须扫描 content blocks。
- Host 应处理 tool execution failure，并让模型看到错误；不要把普通工具失败提升成进程崩溃。

Mini Claude Code s01-s10 | 5

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (150 lines)

```
#!/usr/bin/env python3
"""s01 - Minimal agent loop with one Bash tool.

Goal: expose the core client-tool loop:
    model -> tool_use -> host executes -> tool_result -> model -> ...

Requirements:
    pip install anthropic python-dotenv
    ANTHROPIC_API_KEY=...
    ANTHROPIC_MODEL=<a model id available to your account>
"""
from __future__ import annotations

import json
import os
import subprocess
from typing import Any

from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()

MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()

TOOLS = [
    {
        "name": "bash",
        "description": "Run a shell command in the current project and return stdout/stderr.",
        "input_schema": {
            "type": "object",
            "properties": {
                "command": {"type": "string", "description": "Shell command to execute"}
            },
            "required": ["command"],
            "additionalProperties": False,
        },
    }
]

SYSTEM = """You are Mini Claude Code, a coding agent running in a project directory.
Use the bash tool when you need to inspect files, run tests, or execute commands.
Prefer small, verifiable steps. Explain the final result to the user.
"""


def run_bash(command: str) -> str:
    """Execute a command and return a compact text result."""
    try:
        proc = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"

    payload = {
        "exit_code": proc.returncode,
        "stdout": proc.stdout[-12000:],
        "stderr": proc.stderr[-12000:],
    }
    return json.dumps(payload, ensure_ascii=False)


```

Mini Claude Code s01-s10 | 6

```python
def block_to_dict(block: Any) -> dict[str, Any]:
    """Convert Anthropic SDK content blocks to plain dicts for message history."""
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    if isinstance(block, dict):
        return block
    raise TypeError(f"Unsupported block type: {type(block)!r}")


def render_text(blocks: list[Any]) -> None:
    for block in blocks:
        if getattr(block, "type", None) == "text":
            print(block.text)


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    """Run until Claude stops requesting client tools."""
    while True:
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=SYSTEM,
            tools=TOOLS,
            messages=messages,
        )

        render_text(response.content)
        assistant_content = [block_to_dict(b) for b in response.content]
        messages.append({"role": "assistant", "content": assistant_content})

        if response.stop_reason != "tool_use":
            return messages

        tool_results: list[dict[str, Any]] = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue

            if block.name == "bash":
                result = run_bash(block.input["command"])
            else:
                result = f"ERROR: unknown tool {block.name!r}"

            tool_results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result,
                }
            )

        messages.append({"role": "user", "content": tool_results})


def main() -> None:
    print("Mini Claude Code s01. Type /exit to quit.")
    messages: list[dict[str, Any]] = []

    while True:
        try:
            prompt = input("\nyou> ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break

        if not prompt:
            continue
        if prompt in {"/exit", "/quit"}:
            break

        messages.append({"role": "user", "content": prompt})
        try:
            messages = agent_loop(messages)
        except Exception as exc:  # tutorial-friendly top-level boundary
            print(f"ERROR: {exc}")


if __name__ == "__main__":
    main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 7

README.md (10 lines)

```
0001  # s01 Agent Loop
0002
0003  Run:
0004
0005  ```bash
0006  pip install anthropic python-dotenv
0007  export ANTHROPIC_API_KEY=...
0008  export ANTHROPIC_MODEL=...
0009  python code.py
0010  ```
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 8

3. s02 - Tool Runtime

加入文件工具、工具分发和工作区路径边界

观察项	本阶段
阶段目录	s02_tools
文件数量	2
新增文件	无（主要修改已有文件）
继承/修改文件	README.md, code.py
Python 行数	227

KNOW-WHY：为什么这一层必须存在

- Coding Agent 不能只有万能 shell。结构化文件工具能缩短参数、减少歧义，并让权限系统能够按“读/写/编辑”表达意图。
- Tool Schema 是给概率模型看的 API affordance：名字、描述、参数边界会直接影响模型是否选对工具。
- 工具实现和 Agent Loop 分离后，新增工具不再修改核心循环；这就是后续 Plugin/MCP/Skill 扩展的基础形态。

KNOW-HOW：这一层具体怎么长出来

5. 加入 read_file / write_file / edit_file / glob，并保留 bash 作为通用逃生舱。
6. 用 TOOL_HANDLERS 做 name -> callable 分发，而不是在 Agent Loop 写一长串 if/elif。
7. 所有文件工具先经过 safe_path：resolve 后必须仍位于 WORKSPACE 之内。
8. 对工具输出做截断，避免一次日志或大文件把 Context Window 灌满。

数据流 / 控制流

```
tool_use -> TOOL_HANDLERS[name] -> safe_path/output-limit -> tool_result
```

最容易踩的坑

- 只检查字符串前缀而不 resolve `..`、符号链接和绝对路径，会产生目录穿越。
- edit_file 如果 old_text 有多个匹配，直接替换第一个会导致不可预测修改；本实现强制唯一匹配。
- 把工具返回完整 stdout/stderr 写入上下文，会很快造成上下文污染。

本阶段验收清单

- 读取一个源码文件。
- 尝试写入工作区内临时文件并再读取。
- 尝试访问 `../` 路径，确认被 safe_path 拒绝。

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (227 lines)

```
0001  #!/usr/bin/env python3
0002  """s02 - Add file tools, handler dispatch, and workspace path safety."""
0003  from __future__ import annotations
0004
0005  import glob as globlib
0006  import json
```

```
import os
import subprocess
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
]

SYSTEM = """You are Mini Claude Code, a coding agent.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
"""


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        proc = subprocess.run(
            command,
```

Mini Claude Code s01-s10 | 10

```
            shell=True,
            text=True,
            capture_output=True,
            cwd=WORKSPACE,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    return json.dumps(
        {
            "exit_code": proc.returncode,
            "stdout": truncate(proc.stdout),
            "stderr": truncate(proc.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"


def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
```

Mini Claude Code s01-s10 | 11

```
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=SYSTEM,
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

        messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        if response.stop_reason != "tool_use":
            return messages

        results = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            print(f"[tool] {block.name} {json.dumps(block.input, ensure_ascii=False)[:300]}")
            content, is_error = execute_tool(block.name, dict(block.input))
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        messages.append({"role": "user", "content": results})


def main() -> None:
    print(f"Mini Claude Code s02 - workspace: {WORKSPACE}")
    messages: list[dict[str, Any]] = []
    while True:
        try:
            prompt = input("\nyou> ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if not prompt:
            continue
        if prompt in {"/exit", "/quit"}:
            break
        messages.append({"role": "user", "content": prompt})
        try:
            messages = agent_loop(messages)
        except Exception as exc:
            print(f"ERROR: {exc}")


if __name__ == "__main__":
    main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (10 lines)

```
# s01 Agent Loop

Run:

```bash
pip install anthropic python-dotenv
export ANTHROPIC_API_KEY=...
export ANTHROPIC_MODEL=...
python code.py
```
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 12

4. s03 - Permission System

把副作用控制从工具实现中剥离出来

观察项	本阶段
阶段目录	s03_permissions
文件数量	2
新增文件	无（主要修改已有文件）
继承/修改文件	README.md, code.py
Python 行数	307

KNOW-WHY：为什么这一层必须存在

- “工具存在”不等于“任何时候都允许执行”。Tool capability 与 Permission policy 应是两个独立层。
- 如果每个敏感动作都让模型自己判断风险，安全策略会变成概率性的；Host 侧 Policy Engine 能提供确定性的 deny/ask/allow。
- 审批不是越多越安全。真正目标是把低风险重复操作自动化，把高影响动作升级为显式批准，把明确危险行为直接拒绝。

KNOW-HOW：这一层具体怎么长出来

9. Gate 1：hard deny，命中直接拒绝。
10. Gate 2：规则匹配；读取与已知测试命令自动允许，修改和未知 shell 默认 ask。
11. Gate 3：交互式 allow-once。
12. 权限函数返回“决定 + 原因”，让日志和后续工具结果都有可解释性。

数据流 / 控制流

```
tool_use -> hard deny -> rule match -> optional user approval -> execute -> result
```

最容易踩的坑

- 把 deny list 当完整安全边界。它只能是防御的一层，不能覆盖未知命令组合。
- 把 permission 逻辑写进每个工具处理器，会导致策略散落、难以审计。
- 在非交互环境复用同一审批函数而不定义 fallback，会导致 CI/worker 卡死；s10 会补上 noninteractive policy。

本阶段验收清单

- 只读工具应直接执行。
- 文件写入应请求批准。
- 非 allowlist shell 应请求批准，hard deny 模式应直接拒绝。

为什么 Permission 和 Tool Schema 不能合并

Tool Schema 回答“能力是什么、参数长什么样”；Permission 回答“在当前上下文、当前用户、当前副作用等级下是否允许调用”。同一个 write_file 工具可以在只读审查会话中被 deny，在交互式开发中 ask，在隔离 worktree worker 中 allow。把这两层解耦后，策略可以独立演化。

Mini Claude Code s01-s10 | 13

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (307 lines)

```
#!/usr/bin/env python3
"""s03 - Add a three-gate permission system before tool execution."""
from __future__ import annotations

import glob as globlib
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
]

SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
"""


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
```

Mini Claude Code s01-s10 | 14

```
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        proc = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True,
            cwd=WORKSPACE,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    return json.dumps(
        {
            "exit_code": proc.returncode,
            "stdout": truncate(proc.stdout),
            "stderr": truncate(proc.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"


def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"




class Decision(str, Enum):
    ALLOW = "allow"
```

Mini Claude Code s01-s10 | 15

```
    ASK = "ask"
    DENY = "deny"


# Gate 1: patterns that should never be auto-approved by this tutorial runtime.
HARD_DENY_SUBSTRINGS = (
    "sudo ",
    "chmod -R 777 /",
    "> /dev/sd",
    "mkfs.",
)

# Gate 2: commands considered low-risk and repeatable in a coding workspace.
SAFE_COMMAND_PREFIXES = (
    "pwd", "ls", "find ", "git status", "git diff", "git log",
    "python -m pytest", "pytest", "npm test", "npm run test",
    "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
)


def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
    """Return a policy decision and human-readable reason.

    Order matters: hard deny -> explicit rules -> ask fallback.
    """
    if name in {"read_file", "glob"}:
        return Decision.ALLOW, "read-only workspace operation"

    if name in {"write_file", "edit_file"}:
        try:
            safe_path(args.get("path", ""))
        except Exception as exc:
            return Decision.DENY, str(exc)
        return Decision.ASK, "file modification inside workspace"

    if name == "bash":
        command = str(args.get("command", "")).strip()
        lowered = command.lower()
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
        if answer in {"", "n", "no"}:
            return False


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
```

Mini Claude Code s01-s10 | 16

```
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=SYSTEM,
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

        messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        if response.stop_reason != "tool_use":
            return messages

        results = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
            allowed, permission_reason = check_permission(block.name, args)
            if allowed:
                content, is_error = execute_tool(block.name, args)
            else:
                content, is_error = permission_reason, True
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        messages.append({"role": "user", "content": results})


def main() -> None:
    print(f"Mini Claude Code s03 - workspace: {WORKSPACE}")
    messages: list[dict[str, Any]] = []
    while True:
        try:
            prompt = input("\nyou> ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if not prompt:
            continue
        if prompt in {"/exit", "/quit"}:
            break
        messages.append({"role": "user", "content": prompt})
        try:
            messages = agent_loop(messages)
        except Exception as exc:
            print(f"ERROR: {exc}")


if __name__ == "__main__":
    main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 17

README.md (8 lines)

```
0001  # s03 Permissions
0002
0003  Adds three gates before execution:
0004  1. hard deny
0005  2. rule matching
0006  3. user approval
0007
0008  Run the same way as s02.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 18

5. s04 - Context Management

管理有限上下文：统计、清理与压缩

观察项	本阶段
阶段目录	s04_context
文件数量	3
新增文件	context_manager.py
继承/修改文件	README.md, code.py
Python 行数	409

KNOW-WHY：为什么这一层必须存在

- Agent 的主要稀缺资源不是“消息条数”，而是每次推理可见的有限上下文。文件内容、工具输出、历史讨论都争夺同一个窗口。
- 当上下文越来越长，模型并不会自动知道哪些旧细节可以丢弃。需要 Runtime 主动清理或压缩低价值历史。
- Compaction 的目标不是“把所有内容缩短”，而是把原始轨迹转成可继续执行的高价值状态。

KNOW-HOW：这一层具体怎么长出来

13. 使用字符数/4 做教学版 approximate token 估算，并提供 /context。
14. 当超过预算时，保留最近若干消息，把更早历史交给独立 summarizer 压缩。
15. 摘要只保留目标、约束、发现、修改、测试、未解决问题；然后作为 [COMPACTED_CONTEXT] 重新进入历史。
16. 提供 /clear 和 /compact，让用户可以主动管理上下文。

数据流 / 控制流

```
history -> estimate -> (over budget?) -> summarize old + keep recent -> next model call
```

最容易踩的坑

- 摘要模型如果被要求“总结一切”，容易把大量无关日志也保留下来。
- 把 persistent instruction 只放在对话里，压缩后可能丢失；这正是 s05 引入 CLAUDE.md/Memory 的原因。
- 字符/4 不是精确 tokenizer；它只是一个触发阈值教学实现。

本阶段验收清单

- 降低 MINICLAUDE_CONTEXT_BUDGET，制造自动 compaction。
- 执行 /compact，确认最近消息仍保留。
- 执行 /clear，确认历史归零。

与 Claude Code 公开机制的对应

Claude Code 当前公开文档说明：上下文接近上限时会先清理较老工具输出，再在需要时压缩对话；/context 可以查看占用，/compact 可主动压缩。[R3] 本教程不复制其内部算法，而是用“预算触发 + 结构化摘要 + 保留最近消息”复现同一类工程问题。

Mini Claude Code s01-s10 | 19

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (350 lines)

```
#!/usr/bin/env python3
"""s04 - Add context inspection and model-assisted compaction."""
from __future__ import annotations

import glob as globlib
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv
from context_manager import ContextManager

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000
CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
]

SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
"""


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
```

```
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        proc = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True,
            cwd=WORKSPACE,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    return json.dumps(
        {
            "exit_code": proc.returncode,
            "stdout": truncate(proc.stdout),
            "stderr": truncate(proc.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"


def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"
```

Mini Claude Code s01-s10 | 21

```
class Decision(str, Enum):
    ALLOW = "allow"
    ASK = "ask"
    DENY = "deny"


# Gate 1: patterns that should never be auto-approved by this tutorial runtime.
HARD_DENY_SUBSTRINGS = (
    "sudo ",
    "chmod -R 777 /",
    "> /dev/sd",
    "mkfs.",
)

# Gate 2: commands considered low-risk and repeatable in a coding workspace.
SAFE_COMMAND_PREFIXES = (
    "pwd", "ls", "find ", "git status", "git diff", "git log",
    "python -m pytest", "pytest", "npm test", "npm run test",
    "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
)


def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
    """Return a policy decision and human-readable reason.

    Order matters: hard deny -> explicit rules -> ask fallback.
    """
    if name in {"read_file", "glob"}:
        return Decision.ALLOW, "read-only workspace operation"

    if name in {"write_file", "edit_file"}:
        try:
            safe_path(args.get("path", ""))
        except Exception as exc:
            return Decision.DENY, str(exc)
        return Decision.ASK, "file modification inside workspace"

    if name == "bash":
        command = str(args.get("command", "")).strip()
        lowered = command.lower()
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
        if answer in {"", "n", "no"}:
            return False


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
}


```

```
def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True




def summarize_history(old_messages: list[dict[str, Any]]) -> str:
    """Ask the model to compress old state without tools."""
    prompt = """Summarize this coding-agent conversation for continuation.
Preserve only durable state:
- user goal and constraints
- repository facts discovered
- files changed and why
- commands/tests run and outcomes
- unresolved hypotheses, TODOs, and errors
- important approvals or denied operations
Do not invent facts. Be concise but concrete.

HISTORY JSON:
""" + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
    response = client.messages.create(
        model=MODEL,
        max_tokens=1800,
        system="You compress coding-agent state for a future model turn.",
        messages=[{"role": "user", "content": prompt}],
    )
    return "\n".join(
        block.text for block in response.content if getattr(block, "type", None) == "text"
    )


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        if CONTEXT.needs_compaction(messages):
            print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
            messages = CONTEXT.compact(messages, summarize_history)
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=SYSTEM,
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

        messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        if response.stop_reason != "tool_use":
            return messages

        results = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
            allowed, permission_reason = check_permission(block.name, args)
            if allowed:
                content, is_error = execute_tool(block.name, args)
            else:
                content, is_error = permission_reason, True
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
```

Mini Claude Code s01-s10 | 23

```
                    "is_error": is_error,
                }
            )
        messages.append({"role": "user", "content": results})


def main() -> None:
    print(f"Mini Claude Code s04 - workspace: {WORKSPACE}")
    messages: list[dict[str, Any]] = []
    while True:
        try:
            prompt = input("\nyou> ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if not prompt:
            continue
        if prompt in {"/exit", "/quit"}:
            break
        if prompt == "/context":
            print(CONTEXT.report(messages))
            continue
        if prompt == "/compact":
            messages = CONTEXT.compact(messages, summarize_history)
            print("[context] " + CONTEXT.report(messages))
            continue
        if prompt == "/clear":
            messages = []
            print("[context] cleared")
            continue
        messages.append({"role": "user", "content": prompt})
        try:
            messages = agent_loop(messages)
        except Exception as exc:
            print(f"ERROR: {exc}")


if __name__ == "__main__":
    main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (3 lines)

```
# s04 Context Management

Adds `/context`, `/compact`, `/clear`, a rough token budget, and model-assisted compaction.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```
"""A small context manager: estimate, inspect, and compact message history."""
from __future__ import annotations

import json
from dataclasses import dataclass
from typing import Any, Callable


@dataclass
class ContextStats:
    message_count: int
    approx_tokens: int
    chars: int


class ContextManager:
    def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
        self.max_approx_tokens = max_approx_tokens
        self.keep_recent_messages = keep_recent_messages

    @staticmethod
    def estimate(messages: list[dict[str, Any]]) -> ContextStats:
        raw = json.dumps(messages, ensure_ascii=False, default=str)
        # Deliberately simple heuristic. Production systems should use model-aware counting.
        return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))

    def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
        return self.estimate(messages).approx_tokens >= self.max_approx_tokens

    def report(self, messages: list[dict[str, Any]]) -> str:
```

Mini Claude Code s01-s10 | 24

```
        stats = self.estimate(messages)
        pct = 100 * stats.approx_tokens / self.max_approx_tokens
        return (
            f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:,}, "
            f"budget={self.max_approx_tokens:,} ({pct:.1f}%)"
        )

    def compact(
        self,
        messages: list[dict[str, Any]],
        summarizer: Callable[[list[dict[str, Any]]], str],
    ) -> list[dict[str, Any]]:
        if len(messages) <= self.keep_recent_messages + 2:
            return messages

        cut = len(messages) - self.keep_recent_messages
        old = messages[:cut]
        recent = messages[cut:]
        summary = summarizer(old)
        compacted = {
            "role": "user",
            "content": (
                "[COMPACTED_CONTEXT]\n"
                "The following is a structured summary of earlier conversation state. "
                "Treat it as historical context, not a new user request.\n\n"
                + summary
            ),
        }
        return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 25

6. s05 - Memory

把跨会话知识拆成显式规则与 Agent 记忆

观察项	本阶段
阶段目录	s05_memory
文件数量	4
新增文件	memory.py
继承/修改文件	README.md, code.py, context_manager.py
Python 行数	486

KNOW-WHY：为什么这一层必须存在

- Context 是短期工作记忆，Memory 是跨会话状态。把两者混在一起会让每个新会话重新解释项目约束。
- 持久信息也应分两类：人类显式规定的“应该怎么做”，以及 Agent 工作中积累的“之前学到了什么”。
- Memory 必须可审计、可编辑、可删除，因此这里选用纯 Markdown 文件而不是隐藏数据库。

KNOW-HOW：这一层具体怎么长出来

17. 读取项目根 CLAUDE.md 作为 project instructions。
18. 把 Agent 自己的持久笔记写到 `.mini_claude/memory/MEMORY.md`，启动时只加载受限前缀。
19. 加入 remember 和 recall_memory 工具；系统提示要求只记录真正跨会话有用的事实。
20. build_system 每一轮重新组装当前 Memory，使文件修改后不必重启进程。

数据流 / 控制流

```
CLAUDE.md + MEMORY.md -> build_system -> every model turn; remember -> durable file
```

最容易踩的坑

- 把每个临时 TODO 都写入 Memory，会形成“记忆垃圾场”。
- 把必须严格执行的安全规则只放 Memory；Memory 是上下文，不是 enforcement。
- 教学版 Memory 存在项目目录中，而正式产品可能采用更复杂的 per-repo/user scope；不要把目录结构当成官方源码实现。

本阶段验收清单

- 建立 CLAUDE.md 并询问项目规则。
- 让 Agent remember 一条构建事实，重启后确认仍可 recall。
- 重复 remember 同一条，确认去重。

Context / CLAUDE.md / Auto Memory 的职责分离

机制	谁写	生命周期	适合内容
Context	对话与工具	当前会话	任务状态/证据
CLAUDE.md	人	跨会话	不变量/约束
Memory	Agent	跨会话	发现/偏好/经验

Mini Claude Code s01-s10 | 26

Claude Code 官方公开文档同样区分 CLAUDE.md 与 auto memory，并强调二者都属于 context，而不是强制策略。[R4]

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (368 lines)

```
#!/usr/bin/env python3
"""s05 - Add project instructions and persistent agent memory."""
from __future__ import annotations

import glob as globlib
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv
from context_manager import ContextManager
from memory import MemoryStore

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000
CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
MEMORY = MemoryStore(WORKSPACE)


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
    tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
    tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
]
```

Mini Claude Code s01-s10 | 27

```

BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
Use remember only for durable facts likely to help future sessions, not transient task state.
"""


def build_system() -> str:
    startup = MEMORY.startup_context()
    return BASE_SYSTEM + ("\n\n" + startup if startup else "")


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        proc = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True,
            cwd=WORKSPACE,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    return json.dumps(
        {
            "exit_code": proc.returncode,
            "stdout": truncate(proc.stdout),
            "stderr": truncate(proc.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"


def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


```

Mini Claude Code s01-s10 | 28

```
def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"




class Decision(str, Enum):
    ALLOW = "allow"
    ASK = "ask"
    DENY = "deny"


# Gate 1: patterns that should never be auto-approved by this tutorial runtime.
HARD_DENY_SUBSTRINGS = (
    "sudo ",
    "chmod -R 777 /",
    "> /dev/sd",
    "mkfs.",
)

# Gate 2: commands considered low-risk and repeatable in a coding workspace.
SAFE_COMMAND_PREFIXES = (
    "pwd", "ls", "find ", "git status", "git diff", "git log",
    "python -m pytest", "pytest", "npm test", "npm run test",
    "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
)


def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
    """Return a policy decision and human-readable reason.

    Order matters: hard deny -> explicit rules -> ask fallback.
    """
    if name in {"read_file", "glob", "recall_memory"}:
        return Decision.ALLOW, "read-only workspace operation"

    if name == "remember":
        return Decision.ALLOW, "writes only to the agent memory store"

    if name in {"write_file", "edit_file"}:
        try:
            safe_path(args.get("path", ""))
        except Exception as exc:
            return Decision.DENY, str(exc)
        return Decision.ASK, "file modification inside workspace"

    if name == "bash":
        command = str(args.get("command", "")).strip()
        lowered = command.lower()
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
        if answer in {"", "n", "no"}:
            return False


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
```

Mini Claude Code s01-s10 | 29

```
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
    "remember": MEMORY.remember,
    "recall_memory": MEMORY.search,
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True




def summarize_history(old_messages: list[dict[str, Any]]) -> str:
    """Ask the model to compress old state without tools."""
    prompt = """Summarize this coding-agent conversation for continuation.
Preserve only durable state:
- user goal and constraints
- repository facts discovered
- files changed and why
- commands/tests run and outcomes
- unresolved hypotheses, TODOs, and errors
- important approvals or denied operations
Do not invent facts. Be concise but concrete.

HISTORY JSON:
""" + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
    response = client.messages.create(
        model=MODEL,
        max_tokens=1800,
        system="You compress coding-agent state for a future model turn.",
        messages=[{"role": "user", "content": prompt}],
    )
    return "\n".join(
        block.text for block in response.content if getattr(block, "type", None) == "text"
    )


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        if CONTEXT.needs_compaction(messages):
            print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
            messages = CONTEXT.compact(messages, summarize_history)
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=build_system(),
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

```

```
0307          messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0308          if response.stop_reason != "tool_use":
0309              return messages
0310
0311          results = []
0312          for block in response.content:
0313              if getattr(block, "type", None) != "tool_use":
0314                  continue
0315              args = dict(block.input)
0316              print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0317              allowed, permission_reason = check_permission(block.name, args)
0318              if allowed:
0319                  content, is_error = execute_tool(block.name, args)
0320              else:
0321                  content, is_error = permission_reason, True
0322              results.append(
0323                  {
0324                      "type": "tool_result",
0325                      "tool_use_id": block.id,
0326                      "content": content,
0327                      "is_error": is_error,
0328                  }
0329              )
0330          messages.append({"role": "user", "content": results})
0331
0332
0333  def main() -> None:
0334      print(f"Mini Claude Code s05 - workspace: {WORKSPACE}")
0335      messages: list[dict[str, Any]] = []
0336      while True:
0337          try:
0338              prompt = input("\nyou> ").strip()
0339          except (EOFError, KeyboardInterrupt):
0340              print()
0341              break
0342          if not prompt:
0343              continue
0344          if prompt in {"/exit", "/quit"}:
0345              break
0346          if prompt == "/context":
0347              print(CONTEXT.report(messages))
0348              continue
0349          if prompt == "/compact":
0350              messages = CONTEXT.compact(messages, summarize_history)
0351              print("[context] " + CONTEXT.report(messages))
0352              continue
0353          if prompt == "/clear":
0354              messages = []
0355              print("[context] cleared")
0356              continue
0357          if prompt == "/memory":
0358              print(MEMORY.load_auto_memory() or "(memory is empty)")
0359              continue
0360          messages.append({"role": "user", "content": prompt})
0361          try:
0362              messages = agent_loop(messages)
0363          except Exception as exc:
0364              print(f"ERROR: {exc}")
0365
0366
0367  if __name__ == "__main__":
0368      main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (7 lines)

```
0001  # s05 Memory
0002
0003  Adds:
0004  - project `CLAUDE.md` loading
0005  - `.mini_claude/memory/MEMORY.md`
0006  - `remember` and `recall_memory` tools
0007  - `/memory`
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```
0001  """A small context manager: estimate, inspect, and compact message history."""
0002  from __future__ import annotations
0003
```

```
import json
from dataclasses import dataclass
from typing import Any, Callable


@dataclass
class ContextStats:
    message_count: int
    approx_tokens: int
    chars: int


class ContextManager:
    def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
        self.max_approx_tokens = max_approx_tokens
        self.keep_recent_messages = keep_recent_messages

    @staticmethod
    def estimate(messages: list[dict[str, Any]]) -> ContextStats:
        raw = json.dumps(messages, ensure_ascii=False, default=str)
        # Deliberately simple heuristic. Production systems should use model-aware counting.
        return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))

    def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
        return self.estimate(messages).approx_tokens >= self.max_approx_tokens

    def report(self, messages: list[dict[str, Any]]) -> str:
        stats = self.estimate(messages)
        pct = 100 * stats.approx_tokens / self.max_approx_tokens
        return (
            f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:,}, "
            f"budget={self.max_approx_tokens:,} ({pct:.1f}%)"
        )

    def compact(
        self,
        messages: list[dict[str, Any]],
        summarizer: Callable[[list[dict[str, Any]]], str],
    ) -> list[dict[str, Any]]:
        if len(messages) <= self.keep_recent_messages + 2:
            return messages

        cut = len(messages) - self.keep_recent_messages
        old = messages[:cut]
        recent = messages[cut:]
        summary = summarizer(old)
        compacted = {
            "role": "user",
            "content": (
                "[COMPACTED_CONTEXT]\n"
                "The following is a structured summary of earlier conversation state. "
                "Treat it as historical context, not a new user request.\n\n"
                + summary
            ),
        }
        return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```
"""Persistent project instructions + agent-written memory for Mini Claude Code."""
from __future__ import annotations

from pathlib import Path
from typing import Iterable


class MemoryStore:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.project_instructions = self.workspace / "CLAUDE.md"
        self.memory_dir = self.workspace / ".mini_claude" / "memory"
        self.memory_file = self.memory_dir / "MEMORY.md"

    def load_project_instructions(self) -> str:
        if not self.project_instructions.exists():
            return ""
        return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]

    def load_auto_memory(self) -> str:
```

```
0021          if not self.memory_file.exists():
0022              return ""
0023          # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024          text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025          return "\n".join(text.splitlines()[:200])[:25000]
0026
0027      def remember(self, note: str) -> str:
0028          note = note.strip()
0029          if not note:
0030              return "ERROR: empty memory note"
0031          self.memory_dir.mkdir(parents=True, exist_ok=True)
0032          existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033          normalized = f"- {note}"
0034          if normalized in existing:
0035              return "OK: note already present"
0036          with self.memory_file.open("a", encoding="utf-8") as fh:
0037              if not existing.endswith("\n"):
0038                  fh.write("\n")
0039              fh.write(normalized + "\n")
0040          return "OK: persisted memory"
0041
0042      def search(self, query: str, limit: int = 20) -> str:
0043          if not self.memory_file.exists():
0044              return "(memory is empty)"
0045          q = query.casefold().strip()
0046          lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047          hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048          selected = list(hits)[:limit]
0049          return "\n".join(selected) if selected else "(no memory matches)"
0050
0051      def startup_context(self) -> str:
0052          parts = []
0053          project = self.load_project_instructions().strip()
0054          memory = self.load_auto_memory().strip()
0055          if project:
0056              parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057          if memory:
0058              parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059          return "\n\n".join(parts)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 33

7. s06 - Skills

用渐进式披露封装可重复 SOP

观察项	本阶段
阶段目录	s06_skills
文件数量	6
新增文件	.mini_claude/skills/verify-change/SKILL.md, skills.py
继承/修改文件	README.md, code.py, context_manager.py, memory.py
Python 行数	569

KNOW-WHY：为什么这一层必须存在

- CLAUDE.md 适合始终成立的事实，不适合每次都加载十几步 SOP。Skills 通过“描述常驻、正文按需加载”降低上下文常驻成本。
- 一个 Skill 不是新 Tool capability，而是对已有工具的可复用操作程序。
- 把重复 Prompt 固化为可版本化 Skill，能把个体经验变成团队可共享的操作资产。

KNOW-HOW：这一层具体怎么长出来

21. 扫描 `.mini_claude/skills/*/SKILL.md`，并兼容 `.claude/skills/*/SKILL.md`。
22. 用 YAML frontmatter 解析 name/description；启动只把 catalog 注入 system。
23. 模型判断相关时调用 use_skill，Runtime 才读取正文并作为 tool_result 返回。
24. Skill 支持相对路径引用其他资源，真正需要时仍由普通文件工具读取。

数据流 / 控制流

```
skill metadata -> system catalog -> use_skill(name) -> SKILL.md body -> model follows procedure
```

最容易踩的坑

- Skill 描述写得太宽，会被频繁误触发；写得太窄，又不会被模型发现。
- 把权限能力塞进 Skill；Skill 只是 instruction，不应绕过 Permission/Sandbox。
- 一次加载过大的 Skill 正文仍会占用上下文，所以需要控制体积并拆支持文件。

本阶段验收清单

- 运行 /skills 查看目录。
- 让 Agent 在完成修改后使用 verify-change Skill。
- 临时新增一个 Skill，重启后确认被发现。

Progressive Disclosure 的 token 经济学

如果有 30 个 1,000-token SOP 全部塞进启动 Prompt，每轮都要支付约 30k token 的常驻成本。Skill catalog 只常驻几十到几百 token 的 name/description；只有真正使用时才加载正文。Claude Code 当前 Skills 文档也明确把“正文只在使用时加载”作为关键特性。[R5]

Mini Claude Code s01-s10 | 34

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (381 lines)

```
#!/usr/bin/env python3
"""s06 - Add progressive-disclosure Skills."""
from __future__ import annotations

import glob as globlib
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv
from context_manager import ContextManager
from memory import MemoryStore
from skills import SkillRegistry

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000
CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
MEMORY = MemoryStore(WORKSPACE)
SKILLS = SkillRegistry(WORKSPACE)


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
    tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
    tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
    tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
]

BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
```

Mini Claude Code s01-s10 | 35

```
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
Use remember only for durable facts likely to help future sessions, not transient task state.
"""


def build_system() -> str:
    startup = MEMORY.startup_context()
    catalog = SKILLS.catalog()
    parts = [BASE_SYSTEM]
    if startup:
        parts.append(startup)
    parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
    parts.append("Load a skill with use_skill only when its procedure is needed.")
    return "\n\n".join(parts)


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        proc = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True,
            cwd=WORKSPACE,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    return json.dumps(
        {
            "exit_code": proc.returncode,
            "stdout": truncate(proc.stdout),
            "stderr": truncate(proc.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"


def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
```

Mini Claude Code s01-s10 | 36

```
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"




class Decision(str, Enum):
    ALLOW = "allow"
    ASK = "ask"
    DENY = "deny"


# Gate 1: patterns that should never be auto-approved by this tutorial runtime.
HARD_DENY_SUBSTRINGS = (
    "sudo ",
    "chmod -R 777 /",
    "> /dev/sd",
    "mkfs.",
)

# Gate 2: commands considered low-risk and repeatable in a coding workspace.
SAFE_COMMAND_PREFIXES = (
    "pwd", "ls", "find ", "git status", "git diff", "git log",
    "python -m pytest", "pytest", "npm test", "npm run test",
    "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
)


def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
    """Return a policy decision and human-readable reason.

    Order matters: hard deny -> explicit rules -> ask fallback.
    """
    if name in {"read_file", "glob", "recall_memory", "use_skill"}:
        return Decision.ALLOW, "read-only workspace operation"

    if name == "remember":
        return Decision.ALLOW, "writes only to the agent memory store"

    if name in {"write_file", "edit_file"}:
        try:
            safe_path(args.get("path", ""))
        except Exception as exc:
            return Decision.DENY, str(exc)
        return Decision.ASK, "file modification inside workspace"

    if name == "bash":
        command = str(args.get("command", "")).strip()
        lowered = command.lower()
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
        if answer in {"", "n", "no"}:
            return False
```

Mini Claude Code s01-s10 | 37

```


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
    "remember": MEMORY.remember,
    "recall_memory": MEMORY.search,
    "use_skill": SKILLS.load,
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True




def summarize_history(old_messages: list[dict[str, Any]]) -> str:
    """Ask the model to compress old state without tools."""
    prompt = """Summarize this coding-agent conversation for continuation.
Preserve only durable state:
- user goal and constraints
- repository facts discovered
- files changed and why
- commands/tests run and outcomes
- unresolved hypotheses, TODOs, and errors
- important approvals or denied operations
Do not invent facts. Be concise but concrete.

HISTORY JSON:
""" + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
    response = client.messages.create(
        model=MODEL,
        max_tokens=1800,
        system="You compress coding-agent state for a future model turn.",
        messages=[{"role": "user", "content": prompt}],
    )
    return "\n".join(
        block.text for block in response.content if getattr(block, "type", None) == "text"
    )


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        if CONTEXT.needs_compaction(messages):
            print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
            messages = CONTEXT.compact(messages, summarize_history)
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=build_system(),
            tools=TOOLS,
            messages=messages,
        )
```

Mini Claude Code s01-s10 | 38

```
0312
0313          for block in response.content:
0314              if getattr(block, "type", None) == "text":
0315                  print(block.text)
0316
0317          messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
0318          if response.stop_reason != "tool_use":
0319              return messages
0320
0321          results = []
0322          for block in response.content:
0323              if getattr(block, "type", None) != "tool_use":
0324                  continue
0325              args = dict(block.input)
0326              print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")
0327              allowed, permission_reason = check_permission(block.name, args)
0328              if allowed:
0329                  content, is_error = execute_tool(block.name, args)
0330              else:
0331                  content, is_error = permission_reason, True
0332              results.append(
0333                  {
0334                      "type": "tool_result",
0335                      "tool_use_id": block.id,
0336                      "content": content,
0337                      "is_error": is_error,
0338                  }
0339              )
0340          messages.append({"role": "user", "content": results})
0341
0342
0343  def main() -> None:
0344      print(f"Mini Claude Code s06 - workspace: {WORKSPACE}")
0345      messages: list[dict[str, Any]] = []
0346      while True:
0347          try:
0348              prompt = input("\nyou> ").strip()
0349          except (EOFError, KeyboardInterrupt):
0350              print()
0351              break
0352          if not prompt:
0353              continue
0354          if prompt in {"/exit", "/quit"}:
0355              break
0356          if prompt == "/context":
0357              print(CONTEXT.report(messages))
0358              continue
0359          if prompt == "/compact":
0360              messages = CONTEXT.compact(messages, summarize_history)
0361              print("[context] " + CONTEXT.report(messages))
0362              continue
0363          if prompt == "/clear":
0364              messages = []
0365              print("[context] cleared")
0366              continue
0367          if prompt == "/memory":
0368              print(MEMORY.load_auto_memory() or "(memory is empty)")
0369              continue
0370          if prompt == "/skills":
0371              print(SKILLS.catalog())
0372              continue
0373          messages.append({"role": "user", "content": prompt})
0374          try:
0375              messages = agent_loop(messages)
0376          except Exception as exc:
0377              print(f"ERROR: {exc}")
0378
0379
0380  if __name__ == "__main__":
0381      main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```
0001  ---
0002  name: verify-change
0003  description: Verify a code change before declaring the task complete.
0004  ---
0005
0006  When implementation appears complete:
```

Mini Claude Code s01-s10 | 39

```
1. inspect `git diff --stat` and `git diff`
2. run the narrowest relevant test first
3. run the broader regression suite if practical
4. run lint/typecheck if the repository has them
5. report exact commands and outcomes
6. flag anything that was not verified
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (6 lines)

```
# s06 Skills

Adds skill discovery from `.mini_claude/skills/*/SKILL.md` and `.claude/skills/*/SKILL.md`.
Only names/descriptions are injected at startup. The body loads through `use_skill` on demand.

Dependency added: `pyyaml`.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```
"""A small context manager: estimate, inspect, and compact message history."""
from __future__ import annotations

import json
from dataclasses import dataclass
from typing import Any, Callable


@dataclass
class ContextStats:
    message_count: int
    approx_tokens: int
    chars: int


class ContextManager:
    def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
        self.max_approx_tokens = max_approx_tokens
        self.keep_recent_messages = keep_recent_messages

    @staticmethod
    def estimate(messages: list[dict[str, Any]]) -> ContextStats:
        raw = json.dumps(messages, ensure_ascii=False, default=str)
        # Deliberately simple heuristic. Production systems should use model-aware counting.
        return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))

    def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
        return self.estimate(messages).approx_tokens >= self.max_approx_tokens

    def report(self, messages: list[dict[str, Any]]) -> str:
        stats = self.estimate(messages)
        pct = 100 * stats.approx_tokens / self.max_approx_tokens
        return (
            f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:,}, "
            f"budget={self.max_approx_tokens:,} ({pct:.1f}%)"
        )

    def compact(
        self,
        messages: list[dict[str, Any]],
        summarizer: Callable[[list[dict[str, Any]]], str],
    ) -> list[dict[str, Any]]:
        if len(messages) <= self.keep_recent_messages + 2:
            return messages

        cut = len(messages) - self.keep_recent_messages
        old = messages[:cut]
        recent = messages[cut:]
        summary = summarizer(old)
        compacted = {
            "role": "user",
            "content": (
                "[COMPACTED_CONTEXT]\n"
                "The following is a structured summary of earlier conversation state. "
                "Treat it as historical context, not a new user request.\n\n"
                + summary
            ),
        }
        return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```
"""Persistent project instructions + agent-written memory for Mini Claude Code."""
from __future__ import annotations

from pathlib import Path
from typing import Iterable


class MemoryStore:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.project_instructions = self.workspace / "CLAUDE.md"
        self.memory_dir = self.workspace / ".mini_claude" / "memory"
        self.memory_file = self.memory_dir / "MEMORY.md"

    def load_project_instructions(self) -> str:
        if not self.project_instructions.exists():
            return ""
        return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]

    def load_auto_memory(self) -> str:
        if not self.memory_file.exists():
            return ""
        # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
        text = self.memory_file.read_text(encoding="utf-8", errors="replace")
        return "\n".join(text.splitlines()[:200])[:25000]

    def remember(self, note: str) -> str:
        note = note.strip()
        if not note:
            return "ERROR: empty memory note"
        self.memory_dir.mkdir(parents=True, exist_ok=True)
        existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
        normalized = f"- {note}"
        if normalized in existing:
            return "OK: note already present"
        with self.memory_file.open("a", encoding="utf-8") as fh:
            if not existing.endswith("\n"):
                fh.write("\n")
            fh.write(normalized + "\n")
        return "OK: persisted memory"

    def search(self, query: str, limit: int = 20) -> str:
        if not self.memory_file.exists():
            return "(memory is empty)"
        q = query.casefold().strip()
        lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
        hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
        selected = list(hits)[:limit]
        return "\n".join(selected) if selected else "(no memory matches)"

    def startup_context(self) -> str:
        parts = []
        project = self.load_project_instructions().strip()
        memory = self.load_auto_memory().strip()
        if project:
            parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
        if memory:
            parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
        return "\n\n".join(parts)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```
"""Progressive-disclosure skills for Mini Claude Code."""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any

import yaml


@dataclass(frozen=True)
class Skill:
    name: str
    description: str
    path: Path
```

Mini Claude Code s01-s10 | 41

```
    body: str
    metadata: dict[str, Any]


class SkillRegistry:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.roots = [
            self.workspace / ".mini_claude" / "skills",
            self.workspace / ".claude" / "skills",  # optional compatibility
        ]

    @staticmethod
    def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
        if not text.startswith("---\n"):
            return {}, text
        end = text.find("\n---\n", 4)
        if end == -1:
            return {}, text
        raw = text[4:end]
        body = text[end + 5 :]
        data = yaml.safe_load(raw) or {}
        if not isinstance(data, dict):
            data = {}
        return data, body

    def discover(self) -> dict[str, Skill]:
        found: dict[str, Skill] = {}
        for root in self.roots:
            if not root.exists():
                continue
            for skill_file in sorted(root.glob("*/SKILL.md")):
                text = skill_file.read_text(encoding="utf-8", errors="replace")
                meta, body = self._split_frontmatter(text)
                name = str(meta.get("name") or skill_file.parent.name)
                description = str(meta.get("description") or "No description provided")
                found[name] = Skill(name, description, skill_file, body.strip(), meta)
        return found

    def catalog(self) -> str:
        skills = self.discover()
        if not skills:
            return "(no skills installed)"
        return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())

    def load(self, name: str) -> str:
        skill = self.discover().get(name)
        if skill is None:
            return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
        # A skill can reference support files by relative path; normal file tools can read them later.
        return (
            f"# Skill: {skill.name}\n"
            f"Source: {skill.path.relative_to(self.workspace)}\n\n"
            f"{skill.body[:30000]}"
        )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 42

8. s07 - Hooks

把“建议”升级为确定性的生命周期控制

观察项	本阶段
阶段目录	s07_hooks
文件数量	9
新增文件	.mini_claude/audit_hook.py, .mini_claude/hooks.json, hooks.py
继承/修改文件	.mini_claude/skills/verify-change/SKILL.md, README.md, code.py, context_manager.py, memory.py, skills.py
Python 行数	690

KNOW-WHY：为什么这一层必须存在

- Prompt/Skill 的本质仍是“模型应该这么做”；Hook 用代码在固定生命周期点执行，适合审计、阻断、格式化和策略注入。
- 把 deterministic automation 放在 Runtime，而不是期待模型永远记得执行，可以减少遗漏和 Prompt 竞争。
- PreToolUse 是副作用发生前最后一个统一拦截点；PostToolUse 是统一审计和后处理点。

KNOW-HOW：这一层具体怎么长出来

25. 从 `.mini_claude/hooks.json` 读取 event -> command 列表。
26. Hook 从 stdin 接收 JSON envelope，并可返回 allow/ask/deny、reason、updated_input、output。
27. 工具执行顺序变成：PreHook -> Permission -> Tool -> PostHook。
28. 同时对 session start/end、pre/post compact 提供事件。

数据流 / 控制流

```
tool_use -> PreHook -> Permission -> Tool -> PostHook -> tool_result
```

最容易踩的坑

- Hook 自己也是可执行代码，配置来源必须可信。
- Hook 卡住会把整个 Agent Loop 卡住，因此必须设置 timeout。
- Hook 和 Permission 冲突时需要明确优先级；本实现 PreHook deny 优先，ask 可升级为人工批准。

本阶段验收清单

- 运行一条工具调用后检查 `.mini_claude/audit.jsonl`。
- 让 Hook 返回 deny，确认工具未执行。
- 执行 /compact，检查 pre/post compact 事件。

Instruction vs Enforcement

设计准则

“最好这么做”放 Prompt/Skill；“一定要发生”放 Hook/Policy/Sandbox。Hook 适合格式化、审计、阻断、资源清理和确定性检查。Claude Code 公开 Hook 生命周期覆盖 tool、session、compact、subagent、worktree 等事件。[R6]

Mini Claude Code s01-s10 | 43

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (408 lines)

```
#!/usr/bin/env python3
"""s07 - Add deterministic lifecycle Hooks."""
from __future__ import annotations

import glob as globlib
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv
from context_manager import ContextManager
from memory import MemoryStore
from skills import SkillRegistry
from hooks import HookManager

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000
CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
MEMORY = MemoryStore(WORKSPACE)
SKILLS = SkillRegistry(WORKSPACE)
HOOKS = HookManager(WORKSPACE)


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
    tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
    tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
    tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
]
```

Mini Claude Code s01-s10 | 44

```

BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
Use remember only for durable facts likely to help future sessions, not transient task state.
"""


def build_system() -> str:
    startup = MEMORY.startup_context()
    catalog = SKILLS.catalog()
    parts = [BASE_SYSTEM]
    if startup:
        parts.append(startup)
    parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
    parts.append("Load a skill with use_skill only when its procedure is needed.")
    return "\n\n".join(parts)


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        proc = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True,
            cwd=WORKSPACE,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    return json.dumps(
        {
            "exit_code": proc.returncode,
            "stdout": truncate(proc.stdout),
            "stderr": truncate(proc.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"


def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
```

Mini Claude Code s01-s10 | 45

```
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"




class Decision(str, Enum):
    ALLOW = "allow"
    ASK = "ask"
    DENY = "deny"


# Gate 1: patterns that should never be auto-approved by this tutorial runtime.
HARD_DENY_SUBSTRINGS = (
    "sudo ",
    "chmod -R 777 /",
    "> /dev/sd",
    "mkfs.",
)

# Gate 2: commands considered low-risk and repeatable in a coding workspace.
SAFE_COMMAND_PREFIXES = (
    "pwd", "ls", "find ", "git status", "git diff", "git log",
    "python -m pytest", "pytest", "npm test", "npm run test",
    "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
)


def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
    """Return a policy decision and human-readable reason.

    Order matters: hard deny -> explicit rules -> ask fallback.
    """
    if name in {"read_file", "glob", "recall_memory", "use_skill"}:
        return Decision.ALLOW, "read-only workspace operation"

    if name == "remember":
        return Decision.ALLOW, "writes only to the agent memory store"

    if name in {"write_file", "edit_file"}:
        try:
            safe_path(args.get("path", ""))
        except Exception as exc:
            return Decision.DENY, str(exc)
        return Decision.ASK, "file modification inside workspace"

    if name == "bash":
        command = str(args.get("command", "")).strip()
        lowered = command.lower()
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
```

Mini Claude Code s01-s10 | 46

```
        if answer in {"", "n", "no"}:
            return False


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
    "remember": MEMORY.remember,
    "recall_memory": MEMORY.search,
    "use_skill": SKILLS.load,
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True




def summarize_history(old_messages: list[dict[str, Any]]) -> str:
    """Ask the model to compress old state without tools."""
    prompt = """Summarize this coding-agent conversation for continuation.
Preserve only durable state:
- user goal and constraints
- repository facts discovered
- files changed and why
- commands/tests run and outcomes
- unresolved hypotheses, TODOs, and errors
- important approvals or denied operations
Do not invent facts. Be concise but concrete.

HISTORY JSON:
""" + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
    response = client.messages.create(
        model=MODEL,
        max_tokens=1800,
        system="You compress coding-agent state for a future model turn.",
        messages=[{"role": "user", "content": prompt}],
    )
    return "\n".join(
        block.text for block in response.content if getattr(block, "type", None) == "text"
    )


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        if CONTEXT.needs_compaction(messages):
            print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
            HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
            messages = CONTEXT.compact(messages, summarize_history)
            HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
```

Mini Claude Code s01-s10 | 47

```
            system=build_system(),
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

        messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        if response.stop_reason != "tool_use":
            return messages

        results = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")

            pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
            if pre.updated_input is not None:
                args = pre.updated_input
            if pre.decision == "deny":
                content, is_error = f"DENIED BY HOOK: {pre.reason}", True
            else:
                if pre.decision == "ask":
                    allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
                    permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
                else:
                    allowed, permission_reason = check_permission(block.name, args)
                if allowed:
                    content, is_error = execute_tool(block.name, args)
                else:
                    content, is_error = permission_reason, True

            post = HOOKS.fire(
                "post_tool_use",
                {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
            )
            if post.output:
                content += "\n[hook output]\n" + post.output.strip()
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        messages.append({"role": "user", "content": results})


def main() -> None:
    print(f"Mini Claude Code s07 - workspace: {WORKSPACE}")
    HOOKS.fire("session_start", {"model": MODEL})
    messages: list[dict[str, Any]] = []
    while True:
        try:
            prompt = input("\nyou> ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if not prompt:
            continue
        if prompt in {"/exit", "/quit"}:
            break
        if prompt == "/context":
            print(CONTEXT.report(messages))
            continue
        if prompt == "/compact":
            HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
            messages = CONTEXT.compact(messages, summarize_history)
            HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
            print("[context] " + CONTEXT.report(messages))
            continue
        if prompt == "/clear":
            messages = []
            print("[context] cleared")
            continue
```

Mini Claude Code s01-s10 | 48

```
0392          if prompt == "/memory":
0393              print(MEMORY.load_auto_memory() or "(memory is empty)")
0394              continue
0395          if prompt == "/skills":
0396              print(SKILLS.catalog())
0397              continue
0398          messages.append({"role": "user", "content": prompt})
0399          try:
0400              messages = agent_loop(messages)
0401          except Exception as exc:
0402              print(f"ERROR: {exc}")
0403
0404      HOOKS.fire("session_end", {"message_count": len(messages)})
0405
0406
0407  if __name__ == "__main__":
0408      main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/audit_hook.py (12 lines)

```
0001  #!/usr/bin/env python3
0002  """Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
0003  import json
0004  import sys
0005  from pathlib import Path
0006
0007  event = json.load(sys.stdin)
0008  path = Path('.mini_claude/audit.jsonl')
0009  path.parent.mkdir(parents=True, exist_ok=True)
0010  with path.open('a', encoding='utf-8') as fh:
0011      fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012  print(json.dumps({"decision": "allow"}))
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 lines)

```
0001  {
0002    "session_start": ["python .mini_claude/audit_hook.py"],
0003    "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004    "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005    "pre_compact": ["python .mini_claude/audit_hook.py"],
0006    "post_compact": ["python .mini_claude/audit_hook.py"],
0007    "session_end": ["python .mini_claude/audit_hook.py"]
0008  }
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```
0001  ---
0002  name: verify-change
0003  description: Verify a code change before declaring the task complete.
0004  ---
0005
0006  When implementation appears complete:
0007  1. inspect `git diff --stat` and `git diff`
0008  2. run the narrowest relevant test first
0009  3. run the broader regression suite if practical
0010  4. run lint/typecheck if the repository has them
0011  5. report exact commands and outcomes
0012  6. flag anything that was not verified
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (10 lines)

```
0001  # s07 Hooks
0002
0003  Adds deterministic lifecycle events configured in `.mini_claude/hooks.json`.
0004  Command hooks receive one JSON object on stdin and may return JSON with:
0005  - `decision`: `allow`, `ask`, `deny`
0006  - `reason`
0007  - `updated_input`
0008  - `output`
0009
0010  Implemented events: session start/end, pre/post tool use, pre/post compact.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```
0001  """A small context manager: estimate, inspect, and compact message history."""
0002  from __future__ import annotations
```

Mini Claude Code s01-s10 | 49

```

import json
from dataclasses import dataclass
from typing import Any, Callable


@dataclass
class ContextStats:
    message_count: int
    approx_tokens: int
    chars: int


class ContextManager:
    def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
        self.max_approx_tokens = max_approx_tokens
        self.keep_recent_messages = keep_recent_messages

    @staticmethod
    def estimate(messages: list[dict[str, Any]]) -> ContextStats:
        raw = json.dumps(messages, ensure_ascii=False, default=str)
        # Deliberately simple heuristic. Production systems should use model-aware counting.
        return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))

    def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
        return self.estimate(messages).approx_tokens >= self.max_approx_tokens

    def report(self, messages: list[dict[str, Any]]) -> str:
        stats = self.estimate(messages)
        pct = 100 * stats.approx_tokens / self.max_approx_tokens
        return (
            f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:,}, "
            f"budget={self.max_approx_tokens:,} ({pct:.1f}%)"
        )

    def compact(
        self,
        messages: list[dict[str, Any]],
        summarizer: Callable[[list[dict[str, Any]]], str],
    ) -> list[dict[str, Any]]:
        if len(messages) <= self.keep_recent_messages + 2:
            return messages

        cut = len(messages) - self.keep_recent_messages
        old = messages[:cut]
        recent = messages[cut:]
        summary = summarizer(old)
        compacted = {
            "role": "user",
            "content": (
                "[COMPACTED_CONTEXT]\n"
                "The following is a structured summary of earlier conversation state. "
                "Treat it as historical context, not a new user request.\n\n"
                + summary
            ),
        }
        return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```

"""Deterministic lifecycle hooks for Mini Claude Code."""
from __future__ import annotations

import json
import subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Any


@dataclass
class HookResult:
    decision: str = "allow"   # allow | deny | ask
    reason: str = ""
    updated_input: dict[str, Any] | None = None
    output: str = ""


class HookManager:
```

```
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.config_path = self.workspace / ".mini_claude" / "hooks.json"

    def _config(self) -> dict[str, Any]:
        if not self.config_path.exists():
            return {}
        try:
            data = json.loads(self.config_path.read_text(encoding="utf-8"))
            return data if isinstance(data, dict) else {}
        except Exception as exc:
            print(f"[hook] invalid config: {exc}")
            return {}

    def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
        commands = self._config().get(event, [])
        if isinstance(commands, str):
            commands = [commands]
        result = HookResult()
        for command in commands:
            if not isinstance(command, str) or not command.strip():
                continue
            envelope = {"event": event, "cwd": str(self.workspace), **payload}
            try:
                proc = subprocess.run(
                    command,
                    shell=True,
                    cwd=self.workspace,
                    input=json.dumps(envelope, ensure_ascii=False),
                    text=True,
                    capture_output=True,
                    timeout=15,
                )
            except subprocess.TimeoutExpired:
                return HookResult(decision="deny", reason=f"hook timed out: {command}")

            if proc.stderr.strip():
                print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
            if proc.returncode != 0:
                return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")

            stdout = proc.stdout.strip()
            if not stdout:
                continue
            try:
                data = json.loads(stdout)
            except json.JSONDecodeError:
                result.output += stdout + "\n"
                continue

            if isinstance(data, dict):
                decision = str(data.get("decision", "allow"))
                if decision in {"allow", "deny", "ask"}:
                    result.decision = decision
                if data.get("reason"):
                    result.reason = str(data["reason"])
                if isinstance(data.get("updated_input"), dict):
                    result.updated_input = data["updated_input"]
                if data.get("output"):
                    result.output += str(data["output"]) + "\n"
                if result.decision == "deny":
                    return result
        return result
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```
"""Persistent project instructions + agent-written memory for Mini Claude Code."""
from __future__ import annotations

from pathlib import Path
from typing import Iterable


class MemoryStore:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.project_instructions = self.workspace / "CLAUDE.md"
        self.memory_dir = self.workspace / ".mini_claude" / "memory"
        self.memory_file = self.memory_dir / "MEMORY.md"
```

```
0014
0015     def load_project_instructions(self) -> str:
0016         if not self.project_instructions.exists():
0017             return ""
0018         return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]
0019
0020     def load_auto_memory(self) -> str:
0021         if not self.memory_file.exists():
0022             return ""
0023         # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
0024         text = self.memory_file.read_text(encoding="utf-8", errors="replace")
0025         return "\n".join(text.splitlines()[:200])[:25000]
0026
0027     def remember(self, note: str) -> str:
0028         note = note.strip()
0029         if not note:
0030             return "ERROR: empty memory note"
0031         self.memory_dir.mkdir(parents=True, exist_ok=True)
0032         existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
0033         normalized = f"- {note}"
0034         if normalized in existing:
0035             return "OK: note already present"
0036         with self.memory_file.open("a", encoding="utf-8") as fh:
0037             if not existing.endswith("\n"):
0038                 fh.write("\n")
0039             fh.write(normalized + "\n")
0040         return "OK: persisted memory"
0041
0042     def search(self, query: str, limit: int = 20) -> str:
0043         if not self.memory_file.exists():
0044             return "(memory is empty)"
0045         q = query.casefold().strip()
0046         lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
0047         hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
0048         selected = list(hits)[:limit]
0049         return "\n".join(selected) if selected else "(no memory matches)"
0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```
0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills",  # optional compatibility
0026         ]
0027
0028     @staticmethod
0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):
```

```
            return {}, text
        end = text.find("\n---\n", 4)
        if end == -1:
            return {}, text
        raw = text[4:end]
        body = text[end + 5 :]
        data = yaml.safe_load(raw) or {}
        if not isinstance(data, dict):
            data = {}
        return data, body

    def discover(self) -> dict[str, Skill]:
        found: dict[str, Skill] = {}
        for root in self.roots:
            if not root.exists():
                continue
            for skill_file in sorted(root.glob("*/SKILL.md")):
                text = skill_file.read_text(encoding="utf-8", errors="replace")
                meta, body = self._split_frontmatter(text)
                name = str(meta.get("name") or skill_file.parent.name)
                description = str(meta.get("description") or "No description provided")
                found[name] = Skill(name, description, skill_file, body.strip(), meta)
        return found

    def catalog(self) -> str:
        skills = self.discover()
        if not skills:
            return "(no skills installed)"
        return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())

    def load(self, name: str) -> str:
        skill = self.discover().get(name)
        if skill is None:
            return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
        # A skill can reference support files by relative path; normal file tools can read them later.
        return (
            f"# Skill: {skill.name}\n"
            f"Source: {skill.path.relative_to(self.workspace)}\n\n"
            f"{skill.body[:30000]}"
        )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 53

9. s08 - Subagents

用独立 Context Window 隔离探索与评审

观察项	本阶段
阶段目录	s08_subagents
文件数量	12
新增文件	.mini_claude/agents/explorer.md, .mini_claude/agents/reviewer.md, agents.py
继承/修改文件	.mini_claude/audit_hook.py, .mini_claude/hooks.json, .mini_claude/skills/verify-change/SKILL.md, README.md, code.py, context_manager.py, hooks.py, memory.py ...
Python 行数	855

KNOW-WHY：为什么这一层必须存在

- Subagent 的最大价值往往不是“并行”，而是 Context Isolation：把大量搜索、日志和代码阅读留在独立窗口，只把结论返回主 Agent。
- 角色化 Subagent 可以配置更小的工具集合、不同模型和 turn budget，从而把能力与风险边界一起下放。
- 让生成者与评审者使用不同 Context，可以降低自我确认偏误。

KNOW-HOW：这一层具体怎么长出来

29. 解析 `.mini_claude/agents/*.md` 的 frontmatter：name、description、tools、model、max_turns。
30. spawn_subagent 创建全新的 message history，注入角色 prompt，并只暴露 allowlist 工具。
31. Subagent 内部仍复用 Permission/Hook/Tool Runtime。
32. 禁止嵌套 spawn_subagent，避免教学实现出现递归爆炸。

数据流 / 控制流

```
main context -> spawn_subagent(task) -> new context + restricted tools -> compact result -> main
```

最容易踩的坑

- 把主会话全部历史复制给 Subagent，会失去 Context Isolation 的主要价值。
- Subagent 工具列表过宽，会使“只读探索者”也具备修改能力。
- 主 Agent 把每个小步骤都委托出去会增加成本和协调开销。

本阶段验收清单

- 用 explorer 调查某个模块并确认不会修改文件。
- 用 reviewer 独立审查 git diff。
- 把 max_turns 调低，观察预算耗尽路径。

为什么结果应该“回摘要”，而不是回完整轨迹

主 Agent 委托代码库探索时，真正需要的通常是“关键文件、调用链、证据、风险”，而不是几十次 grep/read 的原始输出。Subagent 独立 context 能把噪声留在侧边窗口，只返回压缩后的高价值状态；这也是 Claude Code 公开文档强调的 context preservation 价值。[R7]

Mini Claude Code s01-s10 | 54

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (504 lines)

```
#!/usr/bin/env python3
"""s08 - Add context-isolated custom Subagents."""
from __future__ import annotations

import glob as globlib
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv
from context_manager import ContextManager
from memory import MemoryStore
from skills import SkillRegistry
from hooks import HookManager
from agents import AgentRegistry

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000
CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
MEMORY = MemoryStore(WORKSPACE)
SKILLS = SkillRegistry(WORKSPACE)
HOOKS = HookManager(WORKSPACE)
AGENTS = AgentRegistry(WORKSPACE)


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
    tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
    tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
```

Mini Claude Code s01-s10 | 55

```
    tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
    tool(
        "spawn_subagent",
        "Delegate a focused side task to a named subagent with its own context window.",
        {"agent": {"type": "string"}, "task": {"type": "string"}},
        ["agent", "task"],
    ),
]

BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
Use remember only for durable facts likely to help future sessions, not transient task state.
"""


def build_system() -> str:
    startup = MEMORY.startup_context()
    catalog = SKILLS.catalog()
    parts = [BASE_SYSTEM]
    if startup:
        parts.append(startup)
    parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
    parts.append("Load a skill with use_skill only when its procedure is needed.")
    parts.append("<available_subagents>\n" + AGENTS.catalog() + "\n</available_subagents>")
    parts.append("Delegate noisy research or independent review with spawn_subagent when useful.")
    return "\n\n".join(parts)


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        proc = subprocess.run(
            command,
            shell=True,
            text=True,
            capture_output=True,
            cwd=WORKSPACE,
            timeout=30,
        )
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    return json.dumps(
        {
            "exit_code": proc.returncode,
            "stdout": truncate(proc.stdout),
            "stderr": truncate(proc.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
```

Mini Claude Code s01-s10 | 56

```

def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"




class Decision(str, Enum):
    ALLOW = "allow"
    ASK = "ask"
    DENY = "deny"


# Gate 1: patterns that should never be auto-approved by this tutorial runtime.
HARD_DENY_SUBSTRINGS = (
    "sudo ",
    "chmod -R 777 /",
    "> /dev/sd",
    "mkfs.",
)

# Gate 2: commands considered low-risk and repeatable in a coding workspace.
SAFE_COMMAND_PREFIXES = (
    "pwd", "ls", "find ", "git status", "git diff", "git log",
    "python -m pytest", "pytest", "npm test", "npm run test",
    "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
)


def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
    """Return a policy decision and human-readable reason.

    Order matters: hard deny -> explicit rules -> ask fallback.
    """
    if name in {"read_file", "glob", "recall_memory", "use_skill", "spawn_subagent"}:
        return Decision.ALLOW, "read-only workspace operation"

    if name == "remember":
        return Decision.ALLOW, "writes only to the agent memory store"

    if name in {"write_file", "edit_file"}:
        try:
            safe_path(args.get("path", ""))
        except Exception as exc:
            return Decision.DENY, str(exc)
        return Decision.ASK, "file modification inside workspace"

    if name == "bash":
        command = str(args.get("command", "")).strip()
        lowered = command.lower()
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

```

Mini Claude Code s01-s10 | 57

```
    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
        if answer in {"", "n", "no"}:
            return False


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
    "remember": MEMORY.remember,
    "recall_memory": MEMORY.search,
    "use_skill": SKILLS.load,
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True




def run_subagent(agent: str, task: str) -> str:
    definition = AGENTS.get(agent)
    if definition is None:
        return f"ERROR: unknown subagent {agent!r}. Available:\n{AGENTS.catalog()}"

    allowed_names = [name for name in definition.tools if name != "spawn_subagent"]
    tool_defs = [item for item in TOOLS if item["name"] in allowed_names]
    sub_messages: list[dict[str, Any]] = [{"role": "user", "content": task}]
    final_text: list[str] = []

    HOOKS.fire("subagent_start", {"agent": definition.name, "task": task})
    for turn in range(definition.max_turns):
        response = client.messages.create(
            model=definition.model or MODEL,
            max_tokens=3000,
            system=(
                build_system()
                + "\n\n<subagent_role>\n"
                + definition.prompt
                + "\n</subagent_role>\n"
                + "You are a delegated worker. Do not spawn nested subagents. Return a compact result to the parent."
            ),
            tools=tool_defs,
            messages=sub_messages,
```

```
        )
        sub_messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        final_text = [b.text for b in response.content if getattr(b, "type", None) == "text"]
        if response.stop_reason != "tool_use":
            break

        results: list[dict[str, Any]] = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            if block.name not in allowed_names:
                content, is_error = f"DENIED: tool {block.name!r} is not allowed for subagent {definition.name}", True
            else:
                pre = HOOKS.fire(
                    "pre_tool_use",
                    {"tool_name": block.name, "tool_input": args, "subagent": definition.name},
                )
                if pre.updated_input is not None:
                    args = pre.updated_input
                if pre.decision == "deny":
                    content, is_error = f"DENIED BY HOOK: {pre.reason}", True
                else:
                    allowed, reason = check_permission(block.name, args)
                    if allowed:
                        content, is_error = execute_tool(block.name, args)
                    else:
                        content, is_error = reason, True
                HOOKS.fire(
                    "post_tool_use",
                    {
                        "tool_name": block.name,
                        "tool_input": args,
                        "tool_output": content,
                        "is_error": is_error,
                        "subagent": definition.name,
                    },
                )
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        sub_messages.append({"role": "user", "content": results})
    else:
        final_text = [f"Subagent reached max_turns={definition.max_turns} before a clean stop."]

    result = "\n".join(final_text).strip() or "(subagent returned no text)"
    HOOKS.fire("subagent_stop", {"agent": definition.name, "result": result})
    return result


# Register after the function exists; this keeps the generic dispatch table from s02.
TOOL_HANDLERS["spawn_subagent"] = run_subagent

def summarize_history(old_messages: list[dict[str, Any]]) -> str:
    """Ask the model to compress old state without tools."""
    prompt = """Summarize this coding-agent conversation for continuation.
Preserve only durable state:
- user goal and constraints
- repository facts discovered
- files changed and why
- commands/tests run and outcomes
- unresolved hypotheses, TODOs, and errors
- important approvals or denied operations
Do not invent facts. Be concise but concrete.

HISTORY JSON:
""" + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
    response = client.messages.create(
        model=MODEL,
        max_tokens=1800,
        system="You compress coding-agent state for a future model turn.",
        messages=[{"role": "user", "content": prompt}],
    )
    return "\n".join(
        block.text for block in response.content if getattr(block, "type", None) == "text"
```

```
    )


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        if CONTEXT.needs_compaction(messages):
            print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
            HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
            messages = CONTEXT.compact(messages, summarize_history)
            HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=build_system(),
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

        messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        if response.stop_reason != "tool_use":
            return messages

        results = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")

            pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
            if pre.updated_input is not None:
                args = pre.updated_input
            if pre.decision == "deny":
                content, is_error = f"DENIED BY HOOK: {pre.reason}", True
            else:
                if pre.decision == "ask":
                    allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
                    permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
                else:
                    allowed, permission_reason = check_permission(block.name, args)
                if allowed:
                    content, is_error = execute_tool(block.name, args)
                else:
                    content, is_error = permission_reason, True

            post = HOOKS.fire(
                "post_tool_use",
                {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
            )
            if post.output:
                content += "\n[hook output]\n" + post.output.strip()
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        messages.append({"role": "user", "content": results})


def main() -> None:
    print(f"Mini Claude Code s08 - workspace: {WORKSPACE}")
    HOOKS.fire("session_start", {"model": MODEL})
    messages: list[dict[str, Any]] = []
    while True:
        try:
            prompt = input("\nyou> ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if not prompt:
            continue
        if prompt in {"/exit", "/quit"}:
            break
```

```
        if prompt == "/context":
            print(CONTEXT.report(messages))
            continue
        if prompt == "/compact":
            HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
            messages = CONTEXT.compact(messages, summarize_history)
            HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
            print("[context] " + CONTEXT.report(messages))
            continue
        if prompt == "/clear":
            messages = []
            print("[context] cleared")
            continue
        if prompt == "/memory":
            print(MEMORY.load_auto_memory() or "(memory is empty)")
            continue
        if prompt == "/skills":
            print(SKILLS.catalog())
            continue
        if prompt == "/agents":
            print(AGENTS.catalog())
            continue
        messages.append({"role": "user", "content": prompt})
        try:
            messages = agent_loop(messages)
        except Exception as exc:
            print(f"ERROR: {exc}")

    HOOKS.fire("session_end", {"message_count": len(messages)})


if __name__ == "__main__":
    main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/explorer.md (13 lines)

```
---
name: explorer
description: Read-only codebase exploration and evidence gathering.
tools:
  - read_file
  - glob
  - bash
max_turns: 10
---

You are a read-only exploration specialist.
Do not modify files. Trace code paths, gather evidence, and return a concise report with exact file paths.
Prefer search and focused reads over dumping large files.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/reviewer.md (12 lines)

```
---
name: reviewer
description: Review a proposed change for correctness, regressions, and missing tests.
tools:
  - read_file
  - glob
  - bash
max_turns: 10
---

Review independently. Inspect the diff and relevant tests. Return findings ordered by severity, with evidence.
Do not edit files.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/audit_hook.py (12 lines)

```
#!/usr/bin/env python3
"""Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
import json
import sys
from pathlib import Path

event = json.load(sys.stdin)
path = Path('.mini_claude/audit.jsonl')
path.parent.mkdir(parents=True, exist_ok=True)
with path.open('a', encoding='utf-8') as fh:
```

Mini Claude Code s01-s10 | 61

```
0011      fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012  print(json.dumps({"decision": "allow"}))
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 lines)

```
0001  {
0002    "session_start": ["python .mini_claude/audit_hook.py"],
0003    "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004    "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005    "pre_compact": ["python .mini_claude/audit_hook.py"],
0006    "post_compact": ["python .mini_claude/audit_hook.py"],
0007    "session_end": ["python .mini_claude/audit_hook.py"]
0008  }
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```
0001  ---
0002  name: verify-change
0003  description: Verify a code change before declaring the task complete.
0004  ---
0005
0006  When implementation appears complete:
0007  1. inspect `git diff --stat` and `git diff`
0008  2. run the narrowest relevant test first
0009  3. run the broader regression suite if practical
0010  4. run lint/typecheck if the repository has them
0011  5. report exact commands and outcomes
0012  6. flag anything that was not verified
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (11 lines)

```
0001  # s08 Subagents
0002
0003  Adds `.mini_claude/agents/*.md` definitions and a `spawn_subagent` tool.
0004  Each subagent gets:
0005  - independent message history/context
0006  - a focused role prompt
0007  - an allowlist of tools
0008  - a max-turn budget
0009  - no nested subagent spawning
0010
0011  Use `/agents` to list definitions.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

agents.py (69 lines)

```
0001  """Custom subagent definitions for Mini Claude Code."""
0002  from __future__ import annotations
0003
0004  from dataclasses import dataclass
0005  from pathlib import Path
0006  from typing import Any
0007
0008  import yaml
0009
0010
0011  @dataclass(frozen=True)
0012  class AgentDefinition:
0013      name: str
0014      description: str
0015      tools: list[str]
0016      model: str | None
0017      max_turns: int
0018      prompt: str
0019      metadata: dict[str, Any]
0020
0021
0022  class AgentRegistry:
0023      def __init__(self, workspace: Path):
0024          self.workspace = workspace.resolve()
0025          self.root = self.workspace / ".mini_claude" / "agents"
0026
0027      @staticmethod
0028      def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0029          if not text.startswith("---\n"):
0030              return {}, text
0031          end = text.find("\n---\n", 4)
```

```
0032          if end == -1:
0033              return {}, text
0034          meta = yaml.safe_load(text[4:end]) or {}
0035          return (meta if isinstance(meta, dict) else {}), text[end + 5 :].strip()
0036
0037      def discover(self) -> dict[str, AgentDefinition]:
0038          found: dict[str, AgentDefinition] = {}
0039          if not self.root.exists():
0040              return found
0041          for path in sorted(self.root.glob("*.md")):
0042              meta, body = self._split_frontmatter(path.read_text(encoding="utf-8", errors="replace"))
0043              name = str(meta.get("name") or path.stem)
0044              raw_tools = meta.get("tools") or ["read_file", "glob", "bash"]
0045              if isinstance(raw_tools, str):
0046                  raw_tools = [x.strip() for x in raw_tools.split(",") if x.strip()]
0047              tools = [str(x) for x in raw_tools]
0048              found[name] = AgentDefinition(
0049                  name=name,
0050                  description=str(meta.get("description") or "No description provided"),
0051                  tools=tools,
0052                  model=str(meta["model"]) if meta.get("model") else None,
0053                  max_turns=int(meta.get("max_turns", 12)),
0054                  prompt=body,
0055                  metadata=meta,
0056              )
0057          return found
0058
0059      def catalog(self) -> str:
0060          agents = self.discover()
0061          if not agents:
0062              return "(no custom subagents installed)"
0063          return "\n".join(
0064              f"- {name}: {agent.description} [tools={','.join(agent.tools)}]"
0065              for name, agent in agents.items()
0066          )
0067
0068      def get(self, name: str) -> AgentDefinition | None:
0069          return self.discover().get(name)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```
0001  """A small context manager: estimate, inspect, and compact message history."""
0002  from __future__ import annotations
0003
0004  import json
0005  from dataclasses import dataclass
0006  from typing import Any, Callable
0007
0008
0009  @dataclass
0010  class ContextStats:
0011      message_count: int
0012      approx_tokens: int
0013      chars: int
0014
0015
0016  class ContextManager:
0017      def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
0018          self.max_approx_tokens = max_approx_tokens
0019          self.keep_recent_messages = keep_recent_messages
0020
0021      @staticmethod
0022      def estimate(messages: list[dict[str, Any]]) -> ContextStats:
0023          raw = json.dumps(messages, ensure_ascii=False, default=str)
0024          # Deliberately simple heuristic. Production systems should use model-aware counting.
0025          return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))
0026
0027      def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
0028          return self.estimate(messages).approx_tokens >= self.max_approx_tokens
0029
0030      def report(self, messages: list[dict[str, Any]]) -> str:
0031          stats = self.estimate(messages)
0032          pct = 100 * stats.approx_tokens / self.max_approx_tokens
0033          return (
0034              f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:,}, "
0035              f"budget={self.max_approx_tokens:,} ({pct:.1f}%)"
0036          )
0037
0038      def compact(
```

Mini Claude Code s01-s10 | 63

```
        self,
        messages: list[dict[str, Any]],
        summarizer: Callable[[list[dict[str, Any]]], str],
    ) -> list[dict[str, Any]]:
        if len(messages) <= self.keep_recent_messages + 2:
            return messages

        cut = len(messages) - self.keep_recent_messages
        old = messages[:cut]
        recent = messages[cut:]
        summary = summarizer(old)
        compacted = {
            "role": "user",
            "content": (
                "[COMPACTED_CONTEXT]\n"
                "The following is a structured summary of earlier conversation state. "
                "Treat it as historical context, not a new user request.\n\n"
                + summary
            ),
        }
        return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```
"""Deterministic lifecycle hooks for Mini Claude Code."""
from __future__ import annotations

import json
import subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Any


@dataclass
class HookResult:
    decision: str = "allow"   # allow | deny | ask
    reason: str = ""
    updated_input: dict[str, Any] | None = None
    output: str = ""


class HookManager:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.config_path = self.workspace / ".mini_claude" / "hooks.json"

    def _config(self) -> dict[str, Any]:
        if not self.config_path.exists():
            return {}
        try:
            data = json.loads(self.config_path.read_text(encoding="utf-8"))
            return data if isinstance(data, dict) else {}
        except Exception as exc:
            print(f"[hook] invalid config: {exc}")
            return {}

    def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
        commands = self._config().get(event, [])
        if isinstance(commands, str):
            commands = [commands]
        result = HookResult()
        for command in commands:
            if not isinstance(command, str) or not command.strip():
                continue
            envelope = {"event": event, "cwd": str(self.workspace), **payload}
            try:
                proc = subprocess.run(
                    command,
                    shell=True,
                    cwd=self.workspace,
                    input=json.dumps(envelope, ensure_ascii=False),
                    text=True,
                    capture_output=True,
                    timeout=15,
                )
            except subprocess.TimeoutExpired:
                return HookResult(decision="deny", reason=f"hook timed out: {command}")

```

```
            if proc.stderr.strip():
                print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
            if proc.returncode != 0:
                return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")

            stdout = proc.stdout.strip()
            if not stdout:
                continue
            try:
                data = json.loads(stdout)
            except json.JSONDecodeError:
                result.output += stdout + "\n"
                continue

            if isinstance(data, dict):
                decision = str(data.get("decision", "allow"))
                if decision in {"allow", "deny", "ask"}:
                    result.decision = decision
                if data.get("reason"):
                    result.reason = str(data["reason"])
                if isinstance(data.get("updated_input"), dict):
                    result.updated_input = data["updated_input"]
                if data.get("output"):
                    result.output += str(data["output"]) + "\n"
                if result.decision == "deny":
                    return result
        return result
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```
"""Persistent project instructions + agent-written memory for Mini Claude Code."""
from __future__ import annotations

from pathlib import Path
from typing import Iterable


class MemoryStore:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.project_instructions = self.workspace / "CLAUDE.md"
        self.memory_dir = self.workspace / ".mini_claude" / "memory"
        self.memory_file = self.memory_dir / "MEMORY.md"

    def load_project_instructions(self) -> str:
        if not self.project_instructions.exists():
            return ""
        return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]

    def load_auto_memory(self) -> str:
        if not self.memory_file.exists():
            return ""
        # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
        text = self.memory_file.read_text(encoding="utf-8", errors="replace")
        return "\n".join(text.splitlines()[:200])[:25000]

    def remember(self, note: str) -> str:
        note = note.strip()
        if not note:
            return "ERROR: empty memory note"
        self.memory_dir.mkdir(parents=True, exist_ok=True)
        existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
        normalized = f"- {note}"
        if normalized in existing:
            return "OK: note already present"
        with self.memory_file.open("a", encoding="utf-8") as fh:
            if not existing.endswith("\n"):
                fh.write("\n")
            fh.write(normalized + "\n")
        return "OK: persisted memory"

    def search(self, query: str, limit: int = 20) -> str:
        if not self.memory_file.exists():
            return "(memory is empty)"
        q = query.casefold().strip()
        lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
        hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
        selected = list(hits)[:limit]
        return "\n".join(selected) if selected else "(no memory matches)"
```

Mini Claude Code s01-s10 | 65

```
0050
0051     def startup_context(self) -> str:
0052         parts = []
0053         project = self.load_project_instructions().strip()
0054         memory = self.load_auto_memory().strip()
0055         if project:
0056             parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
0057         if memory:
0058             parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
0059         return "\n\n".join(parts)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```
0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills",  # optional compatibility
0026         ]
0027
0028     @staticmethod
0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):
0031             return {}, text
0032         end = text.find("\n---\n", 4)
0033         if end == -1:
0034             return {}, text
0035         raw = text[4:end]
0036         body = text[end + 5 :]
0037         data = yaml.safe_load(raw) or {}
0038         if not isinstance(data, dict):
0039             data = {}
0040         return data, body
0041
0042     def discover(self) -> dict[str, Skill]:
0043         found: dict[str, Skill] = {}
0044         for root in self.roots:
0045             if not root.exists():
0046                 continue
0047             for skill_file in sorted(root.glob("*/SKILL.md")):
0048                 text = skill_file.read_text(encoding="utf-8", errors="replace")
0049                 meta, body = self._split_frontmatter(text)
0050                 name = str(meta.get("name") or skill_file.parent.name)
0051                 description = str(meta.get("description") or "No description provided")
0052                 found[name] = Skill(name, description, skill_file, body.strip(), meta)
0053         return found
0054
0055     def catalog(self) -> str:
0056         skills = self.discover()
0057         if not skills:
0058             return "(no skills installed)"
0059         return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())
0060
0061     def load(self, name: str) -> str:
0062         skill = self.discover().get(name)
0063         if skill is None:
0064             return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
0065         # A skill can reference support files by relative path; normal file tools can read them later.
0066         return (
```

Mini Claude Code s01-s10 | 66

```
0067              f"# Skill: {skill.name}\n"
0068              f"Source: {skill.path.relative_to(self.workspace)}\n\n"
0069              f"{skill.body[:30000]}"
0070          )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 67

10. s09 - Sandbox

把模型判断之外再加一层执行边界

观察项	本阶段
阶段目录	s09_sandbox
文件数量	13
新增文件	sandbox.py
继承/修改文件	.mini_claude/agents/explorer.md, .mini_claude/agents/reviewer.md, .mini_claude/audit_hook.py, .mini_claude/hooks.json, .mini_claude/skills/verify-change/SKILL.md, README.md, agents.py, code.py ...
Python 行数	964

KNOW-WHY：为什么这一层必须存在

- Permission 是“这次是否允许做”，Sandbox 是“即使允许，最多能碰到哪里”。两者解决不同问题。
- 模型判断和审批都可能出错，操作系统级 containment 可以把最坏情况限制在更小 blast radius。
- shell 是 Coding Agent 最强也最危险的通用工具，因此先对 shell 建边界最有收益。

KNOW-HOW：这一层具体怎么长出来

33. 新增 SandboxRunner。soft 模式做 cwd 固定、环境变量净化、timeout；明确声明它不是强安全边界。
34. Linux 环境若存在 Bubblewrap，则 bwrap 模式把工作区作为可写目录、系统目录只读，并可 unshare network。
35. Shell 子进程不继承 ANTHROPIC_API_KEY 等敏感环境变量。
36. 加入 sandbox_status 和 /sandbox，避免用户误以为当前处于强隔离。

数据流 / 控制流

```
Permission allow -> SandboxRunner -> sanitized env / cwd / OS boundary -> command result
```

最容易踩的坑

- 把“cwd 在工作区”误认为文件系统隔离；普通进程仍可能访问其他路径。
- Bubblewrap 配置依赖 Linux 发行版，生产环境要针对依赖、设备、网络白名单做更严格设计。
- 网络完全关闭会让 package manager、测试依赖等失败；生产系统通常需要代理/allowlist，而非简单全开。

本阶段验收清单

- 运行 /sandbox 查看 active mode。
- 确认 shell 环境中看不到 API key。
- 在支持 bwrap 的 Linux 上验证网络/路径边界。

Permission 与 Sandbox 的二维模型

层	回答的问题	失后果
Permission	这次调用允许吗？	错误批准可能产生副作用
Sandbox	即使运行，最多能访问哪里？	限制错误批准的 blast radius

Mini Claude Code s01-s10 | 68

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (511 lines)

```
#!/usr/bin/env python3
"""s09 - Add a sandboxed shell execution boundary."""
from __future__ import annotations

import glob as globlib
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv
from context_manager import ContextManager
from memory import MemoryStore
from skills import SkillRegistry
from hooks import HookManager
from agents import AgentRegistry
from sandbox import SandboxRunner

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000
CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
MEMORY = MemoryStore(WORKSPACE)
SKILLS = SkillRegistry(WORKSPACE)
HOOKS = HookManager(WORKSPACE)
AGENTS = AgentRegistry(WORKSPACE)
SANDBOX = SandboxRunner(
    WORKSPACE,
    mode=os.getenv("MINICLAUDE_SANDBOX", "auto"),
    disable_network=os.getenv("MINICLAUDE_ALLOW_NETWORK", "0") != "1",
)


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
```

Mini Claude Code s01-s10 | 69

```
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
    tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
    tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
    tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
    tool(
        "spawn_subagent",
        "Delegate a focused side task to a named subagent with its own context window.",
        {"agent": {"type": "string"}, "task": {"type": "string"}},
        ["agent", "task"],
    ),
    tool("sandbox_status", "Report the active shell sandbox mode.", {}, []),
]

BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
Use remember only for durable facts likely to help future sessions, not transient task state.
"""


def build_system() -> str:
    startup = MEMORY.startup_context()
    catalog = SKILLS.catalog()
    parts = [BASE_SYSTEM]
    if startup:
        parts.append(startup)
    parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
    parts.append("Load a skill with use_skill only when its procedure is needed.")
    parts.append("<available_subagents>\n" + AGENTS.catalog() + "\n</available_subagents>")
    parts.append("Delegate noisy research or independent review with spawn_subagent when useful.")
    return "\n\n".join(parts)


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        result = SANDBOX.run(command, timeout=30)
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    except Exception as exc:
        return f"ERROR: sandbox execution failed: {type(exc).__name__}: {exc}"
    return json.dumps(
        {
            "sandbox": result.mode,
            "exit_code": result.exit_code,
            "stdout": truncate(result.stdout),
            "stderr": truncate(result.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))

```

```
0149
0150 def run_write(path: str, content: str) -> str:
0151     p = safe_path(path)
0152     p.parent.mkdir(parents=True, exist_ok=True)
0153     p.write_text(content, encoding="utf-8")
0154     return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"
0155
0156
0157 def run_edit(path: str, old_text: str, new_text: str) -> str:
0158     p = safe_path(path)
0159     if not p.is_file():
0160         return f"ERROR: file not found: {path}"
0161     text = p.read_text(encoding="utf-8", errors="replace")
0162     count = text.count(old_text)
0163     if count == 0:
0164         return "ERROR: old_text not found"
0165     if count > 1:
0166         return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
0167     p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
0168     return f"OK: edited {p.relative_to(WORKSPACE)}"
0169
0170
0171 def run_glob(pattern: str) -> str:
0172     # Resolve matches then apply safe_path to each result to keep output inside the workspace.
0173     raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
0174     matches: list[str] = []
0175     for raw in raw_matches[:500]:
0176         try:
0177             p = safe_path(raw)
0178         except ValueError:
0179             continue
0180         matches.append(str(p.relative_to(WORKSPACE)))
0181     return "\n".join(sorted(matches)) if matches else "(no matches)"
0182
0183
0184
0185
0186 class Decision(str, Enum):
0187     ALLOW = "allow"
0188     ASK = "ask"
0189     DENY = "deny"
0190
0191
0192 # Gate 1: patterns that should never be auto-approved by this tutorial runtime.
0193 HARD_DENY_SUBSTRINGS = (
0194     "sudo ",
0195     "chmod -R 777 /",
0196     "> /dev/sd",
0197     "mkfs.",
0198 )
0199
0200 # Gate 2: commands considered low-risk and repeatable in a coding workspace.
0201 SAFE_COMMAND_PREFIXES = (
0202     "pwd", "ls", "find ", "git status", "git diff", "git log",
0203     "python -m pytest", "pytest", "npm test", "npm run test",
0204     "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
0205 )
0206
0207
0208 def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
0209     """Return a policy decision and human-readable reason.
0210
0211     Order matters: hard deny -> explicit rules -> ask fallback.
0212     """
0213     if name in {"read_file", "glob", "recall_memory", "use_skill", "spawn_subagent", "sandbox_status"}:
0214         return Decision.ALLOW, "read-only workspace operation"
0215
0216     if name == "remember":
0217         return Decision.ALLOW, "writes only to the agent memory store"
0218
0219     if name in {"write_file", "edit_file"}:
0220         try:
0221             safe_path(args.get("path", ""))
0222         except Exception as exc:
0223             return Decision.DENY, str(exc)
0224         return Decision.ASK, "file modification inside workspace"
0225
0226     if name == "bash":
0227         command = str(args.get("command", "")).strip()
0228         lowered = command.lower()
```

Mini Claude Code s01-s10 | 71

```
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
        if answer in {"", "n", "no"}:
            return False


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
    "remember": MEMORY.remember,
    "recall_memory": MEMORY.search,
    "use_skill": SKILLS.load,
    "sandbox_status": lambda: SANDBOX.status(),
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True




def run_subagent(agent: str, task: str) -> str:
    definition = AGENTS.get(agent)
    if definition is None:
        return f"ERROR: unknown subagent {agent!r}. Available:\n{AGENTS.catalog()}"

    allowed_names = [name for name in definition.tools if name != "spawn_subagent"]
    tool_defs = [item for item in TOOLS if item["name"] in allowed_names]
    sub_messages: list[dict[str, Any]] = [{"role": "user", "content": task}]
    final_text: list[str] = []

    HOOKS.fire("subagent_start", {"agent": definition.name, "task": task})
    for turn in range(definition.max_turns):
        response = client.messages.create(
            model=definition.model or MODEL,
            max_tokens=3000,
            system=(
                build_system()
```

Mini Claude Code s01-s10 | 72

```
                + "\n\n<subagent_role>\n"
                + definition.prompt
                + "\n</subagent_role>\n"
                + "You are a delegated worker. Do not spawn nested subagents. Return a compact result to the parent."
            ),
            tools=tool_defs,
            messages=sub_messages,
        )
        sub_messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        final_text = [b.text for b in response.content if getattr(b, "type", None) == "text"]
        if response.stop_reason != "tool_use":
            break

        results: list[dict[str, Any]] = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            if block.name not in allowed_names:
                content, is_error = f"DENIED: tool {block.name!r} is not allowed for subagent {definition.name}", True
            else:
                pre = HOOKS.fire(
                    "pre_tool_use",
                    {"tool_name": block.name, "tool_input": args, "subagent": definition.name},
                )
                if pre.updated_input is not None:
                    args = pre.updated_input
                if pre.decision == "deny":
                    content, is_error = f"DENIED BY HOOK: {pre.reason}", True
                else:
                    allowed, reason = check_permission(block.name, args)
                    if allowed:
                        content, is_error = execute_tool(block.name, args)
                    else:
                        content, is_error = reason, True
                HOOKS.fire(
                    "post_tool_use",
                    {
                        "tool_name": block.name,
                        "tool_input": args,
                        "tool_output": content,
                        "is_error": is_error,
                        "subagent": definition.name,
                    },
                )
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        sub_messages.append({"role": "user", "content": results})
    else:
        final_text = [f"Subagent reached max_turns={definition.max_turns} before a clean stop."]

    result = "\n".join(final_text).strip() or "(subagent returned no text)"
    HOOKS.fire("subagent_stop", {"agent": definition.name, "result": result})
    return result


# Register after the function exists; this keeps the generic dispatch table from s02.
TOOL_HANDLERS["spawn_subagent"] = run_subagent

def summarize_history(old_messages: list[dict[str, Any]]) -> str:
    """Ask the model to compress old state without tools."""
    prompt = """Summarize this coding-agent conversation for continuation.
Preserve only durable state:
- user goal and constraints
- repository facts discovered
- files changed and why
- commands/tests run and outcomes
- unresolved hypotheses, TODOs, and errors
- important approvals or denied operations
Do not invent facts. Be concise but concrete.

HISTORY JSON:
""" + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
    response = client.messages.create(
```

```
        model=MODEL,
        max_tokens=1800,
        system="You compress coding-agent state for a future model turn.",
        messages=[{"role": "user", "content": prompt}],
    )
    return "\n".join(
        block.text for block in response.content if getattr(block, "type", None) == "text"
    )


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        if CONTEXT.needs_compaction(messages):
            print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
            HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
            messages = CONTEXT.compact(messages, summarize_history)
            HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
        response = client.messages.create(
            model=MODEL,
            max_tokens=4096,
            system=build_system(),
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

        messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        if response.stop_reason != "tool_use":
            return messages

        results = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")

            pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
            if pre.updated_input is not None:
                args = pre.updated_input
            if pre.decision == "deny":
                content, is_error = f"DENIED BY HOOK: {pre.reason}", True
            else:
                if pre.decision == "ask":
                    allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
                    permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
                else:
                    allowed, permission_reason = check_permission(block.name, args)
                if allowed:
                    content, is_error = execute_tool(block.name, args)
                else:
                    content, is_error = permission_reason, True

            post = HOOKS.fire(
                "post_tool_use",
                {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
            )
            if post.output:
                content += "\n[hook output]\n" + post.output.strip()
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        messages.append({"role": "user", "content": results})


def main() -> None:
    print(f"Mini Claude Code s09 - workspace: {WORKSPACE}")
    HOOKS.fire("session_start", {"model": MODEL})
    messages: list[dict[str, Any]] = []
    while True:
        try:
            prompt = input("\nyou> ").strip()
```

Mini Claude Code s01-s10 | 74

```
0469          except (EOFError, KeyboardInterrupt):
0470              print()
0471              break
0472          if not prompt:
0473              continue
0474          if prompt in {"/exit", "/quit"}:
0475              break
0476          if prompt == "/context":
0477              print(CONTEXT.report(messages))
0478              continue
0479          if prompt == "/compact":
0480              HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0481              messages = CONTEXT.compact(messages, summarize_history)
0482              HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
0483              print("[context] " + CONTEXT.report(messages))
0484              continue
0485          if prompt == "/clear":
0486              messages = []
0487              print("[context] cleared")
0488              continue
0489          if prompt == "/memory":
0490              print(MEMORY.load_auto_memory() or "(memory is empty)")
0491              continue
0492          if prompt == "/skills":
0493              print(SKILLS.catalog())
0494              continue
0495          if prompt == "/agents":
0496              print(AGENTS.catalog())
0497              continue
0498          if prompt == "/sandbox":
0499              print(SANDBOX.status())
0500              continue
0501          messages.append({"role": "user", "content": prompt})
0502          try:
0503              messages = agent_loop(messages)
0504          except Exception as exc:
0505              print(f"ERROR: {exc}")
0506
0507      HOOKS.fire("session_end", {"message_count": len(messages)})
0508
0509
0510  if __name__ == "__main__":
0511      main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/explorer.md (13 lines)

```
0001  ---
0002  name: explorer
0003  description: Read-only codebase exploration and evidence gathering.
0004  tools:
0005    - read_file
0006    - glob
0007    - bash
0008  max_turns: 10
0009  ---
0010
0011  You are a read-only exploration specialist.
0012  Do not modify files. Trace code paths, gather evidence, and return a concise report with exact file paths.
0013  Prefer search and focused reads over dumping large files.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/reviewer.md (12 lines)

```
0001  ---
0002  name: reviewer
0003  description: Review a proposed change for correctness, regressions, and missing tests.
0004  tools:
0005    - read_file
0006    - glob
0007    - bash
0008  max_turns: 10
0009  ---
0010
0011  Review independently. Inspect the diff and relevant tests. Return findings ordered by severity, with evidence.
0012  Do not edit files.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 75

.mini_claude/audit_hook.py (12 lines)

```
0001 #!/usr/bin/env python3
0002 """Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
0003 import json
0004 import sys
0005 from pathlib import Path
0006
0007 event = json.load(sys.stdin)
0008 path = Path('.mini_claude/audit.jsonl')
0009 path.parent.mkdir(parents=True, exist_ok=True)
0010 with path.open('a', encoding='utf-8') as fh:
0011     fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012 print(json.dumps({"decision": "allow"}))
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 lines)

```
0001 {
0002   "session_start": ["python .mini_claude/audit_hook.py"],
0003   "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004   "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005   "pre_compact": ["python .mini_claude/audit_hook.py"],
0006   "post_compact": ["python .mini_claude/audit_hook.py"],
0007   "session_end": ["python .mini_claude/audit_hook.py"]
0008 }
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```
0001 ---
0002 name: verify-change
0003 description: Verify a code change before declaring the task complete.
0004 ---
0005
0006 When implementation appears complete:
0007 1. inspect `git diff --stat` and `git diff`
0008 2. run the narrowest relevant test first
0009 3. run the broader regression suite if practical
0010 4. run lint/typecheck if the repository has them
0011 5. report exact commands and outcomes
0012 6. flag anything that was not verified
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (11 lines)

```
0001 # s09 Sandbox
0002
0003 Adds a shell execution boundary:
0004 - `MINICLAUDE_SANDBOX=auto|soft|bwrap`
0005 - sanitized environment (API keys are not passed to commands)
0006 - workspace cwd
0007 - timeout
0008 - optional Bubblewrap filesystem/network isolation on Linux
0009
0010 Important: `soft` mode is a policy convenience, not a strong OS security boundary.
0011 Use `/sandbox` to inspect the active mode.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

agents.py (69 lines)

```
0001 """Custom subagent definitions for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class AgentDefinition:
0013     name: str
0014     description: str
0015     tools: list[str]
0016     model: str | None
0017     max_turns: int
0018     prompt: str
0019     metadata: dict[str, Any]
```

Mini Claude Code s01-s10 | 76

```

class AgentRegistry:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.root = self.workspace / ".mini_claude" / "agents"

    @staticmethod
    def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
        if not text.startswith("---\n"):
            return {}, text
        end = text.find("\n---\n", 4)
        if end == -1:
            return {}, text
        meta = yaml.safe_load(text[4:end]) or {}
        return (meta if isinstance(meta, dict) else {}), text[end + 5 :].strip()

    def discover(self) -> dict[str, AgentDefinition]:
        found: dict[str, AgentDefinition] = {}
        if not self.root.exists():
            return found
        for path in sorted(self.root.glob("*.md")):
            meta, body = self._split_frontmatter(path.read_text(encoding="utf-8", errors="replace"))
            name = str(meta.get("name") or path.stem)
            raw_tools = meta.get("tools") or ["read_file", "glob", "bash"]
            if isinstance(raw_tools, str):
                raw_tools = [x.strip() for x in raw_tools.split(",") if x.strip()]
            tools = [str(x) for x in raw_tools]
            found[name] = AgentDefinition(
                name=name,
                description=str(meta.get("description") or "No description provided"),
                tools=tools,
                model=str(meta["model"]) if meta.get("model") else None,
                max_turns=int(meta.get("max_turns", 12)),
                prompt=body,
                metadata=meta,
            )
        return found

    def catalog(self) -> str:
        agents = self.discover()
        if not agents:
            return "(no custom subagents installed)"
        return "\n".join(
            f"- {name}: {agent.description} [tools={','.join(agent.tools)}]"
            for name, agent in agents.items()
        )

    def get(self, name: str) -> AgentDefinition | None:
        return self.discover().get(name)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```

"""A small context manager: estimate, inspect, and compact message history."""
from __future__ import annotations

import json
from dataclasses import dataclass
from typing import Any, Callable


@dataclass
class ContextStats:
    message_count: int
    approx_tokens: int
    chars: int


class ContextManager:
    def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
        self.max_approx_tokens = max_approx_tokens
        self.keep_recent_messages = keep_recent_messages

    @staticmethod
    def estimate(messages: list[dict[str, Any]]) -> ContextStats:
        raw = json.dumps(messages, ensure_ascii=False, default=str)
        # Deliberately simple heuristic. Production systems should use model-aware counting.
        return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))

```

Mini Claude Code s01-s10 | 77

```
    def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
        return self.estimate(messages).approx_tokens >= self.max_approx_tokens

    def report(self, messages: list[dict[str, Any]]) -> str:
        stats = self.estimate(messages)
        pct = 100 * stats.approx_tokens / self.max_approx_tokens
        return (
            f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:,}, "
            f"budget={self.max_approx_tokens:,} ({pct:.1f}%)"
        )

    def compact(
        self,
        messages: list[dict[str, Any]],
        summarizer: Callable[[list[dict[str, Any]]], str],
    ) -> list[dict[str, Any]]:
        if len(messages) <= self.keep_recent_messages + 2:
            return messages

        cut = len(messages) - self.keep_recent_messages
        old = messages[:cut]
        recent = messages[cut:]
        summary = summarizer(old)
        compacted = {
            "role": "user",
            "content": (
                "[COMPACTED_CONTEXT]\n"
                "The following is a structured summary of earlier conversation state. "
                "Treat it as historical context, not a new user request.\n\n"
                + summary
            ),
        }
        return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```
"""Deterministic lifecycle hooks for Mini Claude Code."""
from __future__ import annotations

import json
import subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Any


@dataclass
class HookResult:
    decision: str = "allow"   # allow | deny | ask
    reason: str = ""
    updated_input: dict[str, Any] | None = None
    output: str = ""


class HookManager:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.config_path = self.workspace / ".mini_claude" / "hooks.json"

    def _config(self) -> dict[str, Any]:
        if not self.config_path.exists():
            return {}
        try:
            data = json.loads(self.config_path.read_text(encoding="utf-8"))
            return data if isinstance(data, dict) else {}
        except Exception as exc:
            print(f"[hook] invalid config: {exc}")
            return {}

    def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
        commands = self._config().get(event, [])
        if isinstance(commands, str):
            commands = [commands]
        result = HookResult()
        for command in commands:
            if not isinstance(command, str) or not command.strip():
                continue
            envelope = {"event": event, "cwd": str(self.workspace), **payload}
            try:
```

Mini Claude Code s01-s10 | 78

```
                proc = subprocess.run(
                    command,
                    shell=True,
                    cwd=self.workspace,
                    input=json.dumps(envelope, ensure_ascii=False),
                    text=True,
                    capture_output=True,
                    timeout=15,
                )
            except subprocess.TimeoutExpired:
                return HookResult(decision="deny", reason=f"hook timed out: {command}")

            if proc.stderr.strip():
                print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
            if proc.returncode != 0:
                return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")

            stdout = proc.stdout.strip()
            if not stdout:
                continue
            try:
                data = json.loads(stdout)
            except json.JSONDecodeError:
                result.output += stdout + "\n"
                continue

            if isinstance(data, dict):
                decision = str(data.get("decision", "allow"))
                if decision in {"allow", "deny", "ask"}:
                    result.decision = decision
                if data.get("reason"):
                    result.reason = str(data["reason"])
                if isinstance(data.get("updated_input"), dict):
                    result.updated_input = data["updated_input"]
                if data.get("output"):
                    result.output += str(data["output"]) + "\n"
                if result.decision == "deny":
                    return result
        return result
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```
"""Persistent project instructions + agent-written memory for Mini Claude Code."""
from __future__ import annotations

from pathlib import Path
from typing import Iterable


class MemoryStore:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.project_instructions = self.workspace / "CLAUDE.md"
        self.memory_dir = self.workspace / ".mini_claude" / "memory"
        self.memory_file = self.memory_dir / "MEMORY.md"

    def load_project_instructions(self) -> str:
        if not self.project_instructions.exists():
            return ""
        return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]

    def load_auto_memory(self) -> str:
        if not self.memory_file.exists():
            return ""
        # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
        text = self.memory_file.read_text(encoding="utf-8", errors="replace")
        return "\n".join(text.splitlines()[:200])[:25000]

    def remember(self, note: str) -> str:
        note = note.strip()
        if not note:
            return "ERROR: empty memory note"
        self.memory_dir.mkdir(parents=True, exist_ok=True)
        existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
        normalized = f"- {note}"
        if normalized in existing:
            return "OK: note already present"
        with self.memory_file.open("a", encoding="utf-8") as fh:
            if not existing.endswith("\n"):
```

Mini Claude Code s01-s10 | 79

```
                fh.write("\n")
            fh.write(normalized + "\n")
        return "OK: persisted memory"

    def search(self, query: str, limit: int = 20) -> str:
        if not self.memory_file.exists():
            return "(memory is empty)"
        q = query.casefold().strip()
        lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
        hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
        selected = list(hits)[:limit]
        return "\n".join(selected) if selected else "(no memory matches)"

    def startup_context(self) -> str:
        parts = []
        project = self.load_project_instructions().strip()
        memory = self.load_auto_memory().strip()
        if project:
            parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
        if memory:
            parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
        return "\n\n".join(parts)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

sandbox.py (102 lines)

```
"""Execution boundary for shell tools.

`soft` mode is portability-oriented and is NOT a strong OS security boundary.
`bwrap` mode uses Bubblewrap on Linux when available and can isolate network + filesystem.
"""
from __future__ import annotations

import os
import shutil
import subprocess
import tempfile
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True)
class SandboxResult:
    exit_code: int
    stdout: str
    stderr: str
    mode: str


class SandboxRunner:
    def __init__(self, workspace: Path, *, mode: str = "auto", disable_network: bool = True):
        self.workspace = workspace.resolve()
        self.requested_mode = mode
        self.disable_network = disable_network
        self.bwrap = shutil.which("bwrap")

    @property
    def mode(self) -> str:
        if self.requested_mode == "auto":
            return "bwrap" if self.bwrap else "soft"
        if self.requested_mode == "bwrap" and not self.bwrap:
            raise RuntimeError("MINICLAUDE_SANDBOX=bwrap requested but `bwrap` is not installed")
        if self.requested_mode not in {"soft", "bwrap"}:
            raise ValueError("sandbox mode must be auto, soft, or bwrap")
        return self.requested_mode

    def sanitized_env(self) -> dict[str, str]:
        allowed = {"PATH", "LANG", "LC_ALL", "TERM", "SHELL", "USER"}
        env = {k: v for k, v in os.environ.items() if k in allowed}
        env["HOME"] = str(self.workspace / ".mini_claude" / "sandbox_home")
        env["MINICLAUDE_SANDBOX"] = self.mode
        Path(env["HOME"]).mkdir(parents=True, exist_ok=True)
        return env

    def run(self, command: str, *, timeout: int = 30) -> SandboxResult:
        mode = self.mode
        env = self.sanitized_env()
        if mode == "soft":
            proc = subprocess.run(
                ["bash", "-lc", command],
```

```
                cwd=self.workspace,
                env=env,
                text=True,
                capture_output=True,
                timeout=timeout,
            )
            return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)

        # Bubblewrap: workspace is writable; common system paths are read-only.
        # This is intentionally conservative and may need distro-specific adjustments.
        tmp = tempfile.mkdtemp(prefix="mini-claude-")
        argv = [
            self.bwrap or "bwrap",
            "--die-with-parent",
            "--new-session",
            "--proc", "/proc",
            "--dev", "/dev",
            "--tmpfs", "/tmp",
            "--bind", str(self.workspace), str(self.workspace),
            "--chdir", str(self.workspace),
        ]
        for system_path in ("/usr", "/bin", "/lib", "/lib64", "/etc"):
            if Path(system_path).exists():
                argv += ["--ro-bind", system_path, system_path]
        if self.disable_network:
            argv.append("--unshare-net")
        argv += ["--setenv", "HOME", str(self.workspace / ".mini_claude" / "sandbox_home")]
        argv += ["bash", "-lc", command]

        try:
            proc = subprocess.run(
                argv,
                cwd=self.workspace,
                env=env,
                text=True,
                capture_output=True,
                timeout=timeout,
            )
            return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)
        finally:
            shutil.rmtree(tmp, ignore_errors=True)

    def status(self) -> str:
        return (
            f"requested={self.requested_mode}, active={self.mode}, "
            f"network={'disabled' if self.disable_network and self.mode == 'bwrap' else 'not strongly isolated'}, "
            f"workspace={self.workspace}"
        )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

skills.py (70 lines)

```
"""Progressive-disclosure skills for Mini Claude Code."""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any

import yaml


@dataclass(frozen=True)
class Skill:
    name: str
    description: str
    path: Path
    body: str
    metadata: dict[str, Any]


class SkillRegistry:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.roots = [
            self.workspace / ".mini_claude" / "skills",
            self.workspace / ".claude" / "skills",  # optional compatibility
        ]

    @staticmethod
```

Mini Claude Code s01-s10 | 81

```
    def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
        if not text.startswith("---\n"):
            return {}, text
        end = text.find("\n---\n", 4)
        if end == -1:
            return {}, text
        raw = text[4:end]
        body = text[end + 5 :]
        data = yaml.safe_load(raw) or {}
        if not isinstance(data, dict):
            data = {}
        return data, body

    def discover(self) -> dict[str, Skill]:
        found: dict[str, Skill] = {}
        for root in self.roots:
            if not root.exists():
                continue
            for skill_file in sorted(root.glob("*/SKILL.md")):
                text = skill_file.read_text(encoding="utf-8", errors="replace")
                meta, body = self._split_frontmatter(text)
                name = str(meta.get("name") or skill_file.parent.name)
                description = str(meta.get("description") or "No description provided")
                found[name] = Skill(name, description, skill_file, body.strip(), meta)
        return found

    def catalog(self) -> str:
        skills = self.discover()
        if not skills:
            return "(no skills installed)"
        return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())

    def load(self, name: str) -> str:
        skill = self.discover().get(name)
        if skill is None:
            return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
        # A skill can reference support files by relative path; normal file tools can read them later.
        return (
            f"# Skill: {skill.name}\n"
            f"Source: {skill.path.relative_to(self.workspace)}\n\n"
            f"{skill.body[:30000]}"
        )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 82

11. s10 - Multi-Agent Runtime

Worktree + 并行 Worker + Mailbox 的最小多 Agent Runtime

观察项	本阶段
阶段目录	s10_multi_agent
文件数量	15
新增文件	.mini_claude/agents/implementer.md, team.py
继承/修改文件	.mini_claude/agents/explorer.md, .mini_claude/agents/reviewer.md, .mini_claude/audit_hook.py, .mini_claude/hooks.json, .mini_claude/skills/verify-change/SKILL.md, README.md, agents.py, code.py ...
Python 行数	1293

KNOW-WHY：为什么这一层必须存在

- 多个 Agent 同时修改同一个 checkout 会产生文件竞争和 Git 状态污染。真正的并行软件工程需要 Context Isolation + Filesystem Isolation + Branch Isolation。
- 多 Agent 不应该等同于“多开几个模型”。需要任务切分、身份、通信、结果收集、资源上限和合并策略。
- 自动 merge 会把并发冲突从执行期推到隐藏的集成期，因此教学 Runtime 选择永不自动合并，只留下 branch/worktree 供 Lead 或人类审查。

KNOW-HOW：这一层具体怎么长出来

37. run_team 接收最多 8 个独立 WorkerSpec，并为每个 worker 创建 `mini/<team>/<worker>` Git branch + worktree。
38. 每个 worker 是独立 Python 子进程，在自己的 cwd 中运行 Mini Claude Code；因此工具路径、安全策略和 Context 都自然隔离。
39. ThreadPoolExecutor 并发运行 worker；工作完成后收集 stdout/stderr、git status、diff stat、branch/worktree。
40. TeamBus 使用共享 JSONL mailbox 提供 team_send/team_inbox，足以演示最小跨 Agent 通信。
41. 非交互 worker 对隔离 worktree 内的 write/edit 自动允许，但未知 shell 仍不会弹窗等待。

数据流 / 控制流

```
Lead -> run_team -> N worktrees -> N worker processes -> mailbox/results -> Lead reviews branches
```

最容易踩的坑

- 并行度越高 Token/API 成本越高，不应把线性任务强行拆成团队。
- 工作任务如果修改同一逻辑区域，即使 worktree 不冲突，最终合并仍会冲突。
- 文件 mailbox 只是教学实现，不处理可靠投递、锁、幂等、背压和实时事件；生产系统应使用更成熟的协调层。
- Worker 进程复用 API key，因此 Host 仍需承担凭据管理；Shell sandbox 会主动剥离这些敏感变量。

本阶段验收清单

- 在 Git repo 中启动两个只读 worker，确认并行完成。
- 让两个 implementer 修改互不重叠文件，确认得到两个独立 branch/worktree。
- 让 worker team_send 一条消息，并由另一 worker team_inbox 读取。
- 确认 Runtime 不会自动 merge。

Mini Claude Code s01-s10 | 83

Subagent、Worktree、Team 三个维度不要混为一谈

机制	隔离对象	是否并行	主要用途
Subagent	Context	可并行/可串行	委托研究/评审
Worktree	Filesystem + branch	支持	避免并发编辑冲突
Team Runtime	Context + workers + coordination	是	拆分可独立任务

Claude Code 当前公开文档把 subagents、worktrees 与 agent teams 明确作为不同并行方式：worktree 隔离文件，subagent 在单会话内委托独立 context，agent teams 增加共享任务与消息协调；Agent Teams 当前文档仍标记 experimental。[R7][R8][R9]

本阶段完整源码

以下列出该阶段目录中所有教学源码/配置文件。为保证“从 s01 到 s10 每一步都能落地”，这里保留完整文件，而不仅仅给出 diff。

code.py (580 lines)

```
#!/usr/bin/env python3
"""s10 - Add a worktree-isolated parallel Multi-Agent Runtime."""
from __future__ import annotations

import glob as globlib
import argparse
import json
import os
import subprocess
import shlex
from enum import Enum
from pathlib import Path
from typing import Any, Callable

from anthropic import Anthropic
from dotenv import load_dotenv
from context_manager import ContextManager
from memory import MemoryStore
from skills import SkillRegistry
from hooks import HookManager
from agents import AgentRegistry
from sandbox import SandboxRunner
from team import TeamBus, TeamRuntime

load_dotenv()
MODEL = os.environ.get("ANTHROPIC_MODEL")
if not MODEL:
    raise RuntimeError("Set ANTHROPIC_MODEL to a model ID available to your account")

client = Anthropic()
WORKSPACE = Path.cwd().resolve()
MAX_TOOL_OUTPUT = 12000
CONTEXT = ContextManager(max_approx_tokens=int(os.getenv("MINICLAUDE_CONTEXT_BUDGET", "24000")))
MEMORY = MemoryStore(WORKSPACE)
SKILLS = SkillRegistry(WORKSPACE)
HOOKS = HookManager(WORKSPACE)
AGENTS = AgentRegistry(WORKSPACE)
SANDBOX = SandboxRunner(
    WORKSPACE,
    mode=os.getenv("MINICLAUDE_SANDBOX", "auto"),
    disable_network=os.getenv("MINICLAUDE_ALLOW_NETWORK", "0") != "1",
)
TEAM_BUS = TeamBus()
TEAM = TeamRuntime(WORKSPACE, Path(__file__))


def tool(name: str, description: str, properties: dict[str, Any], required: list[str]) -> dict[str, Any]:
    return {
        "name": name,
        "description": description,
        "input_schema": {
```

Mini Claude Code s01-s10 | 84

```
            "type": "object",
            "properties": properties,
            "required": required,
            "additionalProperties": False,
        },
    }


TOOLS = [
    tool("bash", "Run a shell command in the workspace.", {"command": {"type": "string"}}, ["command"]),
    tool("read_file", "Read a UTF-8 text file from the workspace.", {"path": {"type": "string"}}, ["path"]),
    tool(
        "write_file",
        "Create or replace a UTF-8 text file inside the workspace.",
        {"path": {"type": "string"}, "content": {"type": "string"}},
        ["path", "content"],
    ),
    tool(
        "edit_file",
        "Replace one exact text occurrence in a workspace file.",
        {
            "path": {"type": "string"},
            "old_text": {"type": "string"},
            "new_text": {"type": "string"},
        },
        ["path", "old_text", "new_text"],
    ),
    tool("glob", "List workspace files matching a glob pattern.", {"pattern": {"type": "string"}}, ["pattern"]),
    tool("remember", "Persist a durable project learning for future sessions.", {"note": {"type": "string"}}, ["note"]),
    tool("recall_memory", "Search persistent memory notes.", {"query": {"type": "string"}}, ["query"]),
    tool("use_skill", "Load a reusable task procedure by name when it is relevant.", {"name": {"type": "string"}}, ["name"]),
    tool(
        "spawn_subagent",
        "Delegate a focused side task to a named subagent with its own context window.",
        {"agent": {"type": "string"}, "task": {"type": "string"}},
        ["agent", "task"],
    ),
    tool("sandbox_status", "Report the active shell sandbox mode.", {}, []),
    tool("team_send", "Send a concise message to another worker when running inside a team.", {"to": {"type": "string"}, "message": {"type":
"string"}}, ["to", "message"]),
    tool("team_inbox", "Read recent teammate messages when running inside a team.", {}, []),
    tool(
        "run_team",
        "Run independent worker agents in parallel Git worktrees. Use for genuinely separable tasks; results are never auto-merged.",
        {
            "team_name": {"type": "string"},
            "tasks": {
                "type": "array",
                "minItems": 1,
                "maxItems": 8,
                "items": {
                    "type": "object",
                    "properties": {
                        "name": {"type": "string"},
                        "agent": {"type": "string"},
                        "prompt": {"type": "string"}
                    },
                    "required": ["name", "prompt"],
                    "additionalProperties": False
                }
            }
        },
        ["team_name", "tasks"],
    ),
]

BASE_SYSTEM = """You are Mini Claude Code, a coding agent with a permission boundary.
Inspect before editing. Prefer read_file/glob for source inspection and bash for tests/builds.
Make focused changes and verify them before you finish.
Use remember only for durable facts likely to help future sessions, not transient task state.
"""


def build_system() -> str:
    startup = MEMORY.startup_context()
    catalog = SKILLS.catalog()
    parts = [BASE_SYSTEM]
    if startup:
        parts.append(startup)
    parts.append("<available_skills>\n" + catalog + "\n</available_skills>")
```

Mini Claude Code s01-s10 | 85

```
    parts.append("Load a skill with use_skill only when its procedure is needed.")
    parts.append("<available_subagents>\n" + AGENTS.catalog() + "\n</available_subagents>")
    parts.append("Delegate noisy research or independent review with spawn_subagent when useful.")
    team_context = TEAM_BUS.context()
    if team_context:
        parts.append("<team_context>\n" + team_context + "\n</team_context>")
    return "\n\n".join(parts)


def safe_path(raw: str) -> Path:
    """Resolve a path and reject escapes outside the workspace."""
    candidate = (WORKSPACE / raw).resolve() if not Path(raw).is_absolute() else Path(raw).resolve()
    try:
        candidate.relative_to(WORKSPACE)
    except ValueError as exc:
        raise ValueError(f"Path escapes workspace: {raw!r}") from exc
    return candidate


def truncate(text: str, limit: int = MAX_TOOL_OUTPUT) -> str:
    if len(text) <= limit:
        return text
    half = limit // 2
    return text[:half] + "\n...<truncated>...\n" + text[-half:]


def run_bash(command: str) -> str:
    try:
        result = SANDBOX.run(command, timeout=30)
    except subprocess.TimeoutExpired:
        return "ERROR: command timed out after 30 seconds"
    except Exception as exc:
        return f"ERROR: sandbox execution failed: {type(exc).__name__}: {exc}"
    return json.dumps(
        {
            "sandbox": result.mode,
            "exit_code": result.exit_code,
            "stdout": truncate(result.stdout),
            "stderr": truncate(result.stderr),
        },
        ensure_ascii=False,
    )


def run_read(path: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    return truncate(p.read_text(encoding="utf-8", errors="replace"))


def run_write(path: str, content: str) -> str:
    p = safe_path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    p.write_text(content, encoding="utf-8")
    return f"OK: wrote {len(content)} chars to {p.relative_to(WORKSPACE)}"


def run_edit(path: str, old_text: str, new_text: str) -> str:
    p = safe_path(path)
    if not p.is_file():
        return f"ERROR: file not found: {path}"
    text = p.read_text(encoding="utf-8", errors="replace")
    count = text.count(old_text)
    if count == 0:
        return "ERROR: old_text not found"
    if count > 1:
        return f"ERROR: old_text is ambiguous ({count} matches); provide a larger unique snippet"
    p.write_text(text.replace(old_text, new_text, 1), encoding="utf-8")
    return f"OK: edited {p.relative_to(WORKSPACE)}"


def run_glob(pattern: str) -> str:
    # Resolve matches then apply safe_path to each result to keep output inside the workspace.
    raw_matches = globlib.glob(str(WORKSPACE / pattern), recursive=True)
    matches: list[str] = []
    for raw in raw_matches[:500]:
        try:
            p = safe_path(raw)
        except ValueError:
```

Mini Claude Code s01-s10 | 86

```
            continue
        matches.append(str(p.relative_to(WORKSPACE)))
    return "\n".join(sorted(matches)) if matches else "(no matches)"




class Decision(str, Enum):
    ALLOW = "allow"
    ASK = "ask"
    DENY = "deny"


# Gate 1: patterns that should never be auto-approved by this tutorial runtime.
HARD_DENY_SUBSTRINGS = (
    "sudo ",
    "chmod -R 777 /",
    "> /dev/sd",
    "mkfs.",
)

# Gate 2: commands considered low-risk and repeatable in a coding workspace.
SAFE_COMMAND_PREFIXES = (
    "pwd", "ls", "find ", "git status", "git diff", "git log",
    "python -m pytest", "pytest", "npm test", "npm run test",
    "npm run lint", "ruff ", "mypy ", "go test", "cargo test",
)


def classify_permission(name: str, args: dict[str, Any]) -> tuple[Decision, str]:
    """Return a policy decision and human-readable reason.

    Order matters: hard deny -> explicit rules -> ask fallback.
    """
    if name in {"read_file", "glob", "recall_memory", "use_skill", "spawn_subagent", "sandbox_status", "team_send", "team_inbox"}:
        return Decision.ALLOW, "read-only or coordination operation"

    if name == "run_team":
        return Decision.ASK, "spawns multiple model workers and creates Git worktrees"

    if name == "remember":
        return Decision.ALLOW, "writes only to the agent memory store"

    if name in {"write_file", "edit_file"}:
        if os.getenv("MINICLAUDE_WORKER_ALLOW_EDITS") == "1":
            return Decision.ALLOW, "isolated team worker edit policy"
        try:
            safe_path(args.get("path", ""))
        except Exception as exc:
            return Decision.DENY, str(exc)
        return Decision.ASK, "file modification inside workspace"

    if name == "bash":
        command = str(args.get("command", "")).strip()
        lowered = command.lower()
        if any(pattern.lower() in lowered for pattern in HARD_DENY_SUBSTRINGS):
            return Decision.DENY, "matches hard deny policy"
        if command.startswith(SAFE_COMMAND_PREFIXES):
            return Decision.ALLOW, "matches low-risk command allowlist"
        return Decision.ASK, "shell command is not allowlisted"

    return Decision.ASK, "unknown or extensible tool"


def request_approval(name: str, args: dict[str, Any], reason: str) -> bool:
    if os.getenv("MINICLAUDE_NONINTERACTIVE") == "1":
        print(f"[permission] noninteractive denial for {name}: {reason}")
        return False
    print(f"\n[permission] {name}: {reason}")
    print(json.dumps(args, ensure_ascii=False, indent=2)[:2000])
    while True:
        answer = input("Allow once? [y/N] ").strip().lower()
        if answer in {"y", "yes"}:
            return True
        if answer in {"", "n", "no"}:
            return False


def check_permission(name: str, args: dict[str, Any]) -> tuple[bool, str]:
    decision, reason = classify_permission(name, args)
```

Mini Claude Code s01-s10 | 87

```
    if decision is Decision.DENY:
        return False, f"DENIED: {reason}"
    if decision is Decision.ALLOW:
        return True, reason
    if request_approval(name, args, reason):
        return True, "approved by user"
    return False, "DENIED: user rejected tool call"


TOOL_HANDLERS: dict[str, Callable[..., str]] = {
    "bash": run_bash,
    "read_file": run_read,
    "write_file": run_write,
    "edit_file": run_edit,
    "glob": run_glob,
    "remember": MEMORY.remember,
    "recall_memory": MEMORY.search,
    "use_skill": SKILLS.load,
    "sandbox_status": lambda: SANDBOX.status(),
    "team_send": TEAM_BUS.send,
    "team_inbox": TEAM_BUS.inbox,
    "run_team": TEAM.run,
}


def block_to_dict(block: Any) -> dict[str, Any]:
    if hasattr(block, "model_dump"):
        return block.model_dump(exclude_none=True)
    return dict(block)


def execute_tool(name: str, args: dict[str, Any]) -> tuple[str, bool]:
    handler = TOOL_HANDLERS.get(name)
    if handler is None:
        return f"Unknown tool: {name}", True
    try:
        return handler(**args), False
    except Exception as exc:
        return f"{type(exc).__name__}: {exc}", True







def run_subagent(agent: str, task: str) -> str:
    definition = AGENTS.get(agent)
    if definition is None:
        return f"ERROR: unknown subagent {agent!r}. Available:\n{AGENTS.catalog()}"

    allowed_names = [name for name in definition.tools if name != "spawn_subagent"]
    tool_defs = [item for item in TOOLS if item["name"] in allowed_names]
    sub_messages: list[dict[str, Any]] = [{"role": "user", "content": task}]
    final_text: list[str] = []

    HOOKS.fire("subagent_start", {"agent": definition.name, "task": task})
    for turn in range(definition.max_turns):
        response = client.messages.create(
            model=definition.model or MODEL,
            max_tokens=3000,
            system=(
                build_system()
                + "\n\n<subagent_role>\n"
                + definition.prompt
                + "\n</subagent_role>\n"
                + "You are a delegated worker. Do not spawn nested subagents. Return a compact result to the parent."
            ),
            tools=tool_defs,
            messages=sub_messages,
        )
        sub_messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        final_text = [b.text for b in response.content if getattr(b, "type", None) == "text"]
        if response.stop_reason != "tool_use":
            break

        results: list[dict[str, Any]] = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            if block.name not in allowed_names:
```

Mini Claude Code s01-s10 | 88

```
                content, is_error = f"DENIED: tool {block.name!r} is not allowed for subagent {definition.name}", True
            else:
                pre = HOOKS.fire(
                    "pre_tool_use",
                    {"tool_name": block.name, "tool_input": args, "subagent": definition.name},
                )
                if pre.updated_input is not None:
                    args = pre.updated_input
                if pre.decision == "deny":
                    content, is_error = f"DENIED BY HOOK: {pre.reason}", True
                else:
                    allowed, reason = check_permission(block.name, args)
                    if allowed:
                        content, is_error = execute_tool(block.name, args)
                    else:
                        content, is_error = reason, True
                HOOKS.fire(
                    "post_tool_use",
                    {
                        "tool_name": block.name,
                        "tool_input": args,
                        "tool_output": content,
                        "is_error": is_error,
                        "subagent": definition.name,
                    },
                )
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        sub_messages.append({"role": "user", "content": results})
    else:
        final_text = [f"Subagent reached max_turns={definition.max_turns} before a clean stop."]

    result = "\n".join(final_text).strip() or "(subagent returned no text)"
    HOOKS.fire("subagent_stop", {"agent": definition.name, "result": result})
    return result


# Register after the function exists; this keeps the generic dispatch table from s02.
TOOL_HANDLERS["spawn_subagent"] = run_subagent

def summarize_history(old_messages: list[dict[str, Any]]) -> str:
    """Ask the model to compress old state without tools."""
    prompt = """Summarize this coding-agent conversation for continuation.
Preserve only durable state:
- user goal and constraints
- repository facts discovered
- files changed and why
- commands/tests run and outcomes
- unresolved hypotheses, TODOs, and errors
- important approvals or denied operations
Do not invent facts. Be concise but concrete.

HISTORY JSON:
""" + json.dumps(old_messages, ensure_ascii=False, default=str)[-60000:]
    response = client.messages.create(
        model=MODEL,
        max_tokens=1800,
        system="You compress coding-agent state for a future model turn.",
        messages=[{"role": "user", "content": prompt}],
    )
    return "\n".join(
        block.text for block in response.content if getattr(block, "type", None) == "text"
    )


def agent_loop(messages: list[dict[str, Any]]) -> list[dict[str, Any]]:
    while True:
        if CONTEXT.needs_compaction(messages):
            print(f"[context] auto-compact before request: {CONTEXT.report(messages)}")
            HOOKS.fire("pre_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
            messages = CONTEXT.compact(messages, summarize_history)
            HOOKS.fire("post_compact", {"reason": "auto", "context": CONTEXT.report(messages)})
        response = client.messages.create(
            model=MODEL,
```

Mini Claude Code s01-s10 | 89

```
            max_tokens=4096,
            system=build_system(),
            tools=TOOLS,
            messages=messages,
        )

        for block in response.content:
            if getattr(block, "type", None) == "text":
                print(block.text)

        messages.append({"role": "assistant", "content": [block_to_dict(b) for b in response.content]})
        if response.stop_reason != "tool_use":
            return messages

        results = []
        for block in response.content:
            if getattr(block, "type", None) != "tool_use":
                continue
            args = dict(block.input)
            print(f"[tool] {block.name} {json.dumps(args, ensure_ascii=False)[:300]}")

            pre = HOOKS.fire("pre_tool_use", {"tool_name": block.name, "tool_input": args})
            if pre.updated_input is not None:
                args = pre.updated_input
            if pre.decision == "deny":
                content, is_error = f"DENIED BY HOOK: {pre.reason}", True
            else:
                if pre.decision == "ask":
                    allowed = request_approval(block.name, args, pre.reason or "hook requested approval")
                    permission_reason = "approved by user" if allowed else "DENIED: user rejected hook approval"
                else:
                    allowed, permission_reason = check_permission(block.name, args)
                if allowed:
                    content, is_error = execute_tool(block.name, args)
                else:
                    content, is_error = permission_reason, True

            post = HOOKS.fire(
                "post_tool_use",
                {"tool_name": block.name, "tool_input": args, "tool_output": content, "is_error": is_error},
            )
            if post.output:
                content += "\n[hook output]\n" + post.output.strip()
            results.append(
                {
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": content,
                    "is_error": is_error,
                }
            )
        messages.append({"role": "user", "content": results})


def run_one_prompt(prompt: str, agent: str | None = None) -> int:
    HOOKS.fire("session_start", {"model": MODEL, "noninteractive": True})
    try:
        if agent:
            print(run_subagent(agent, prompt))
        else:
            messages: list[dict[str, Any]] = [{"role": "user", "content": prompt}]
            agent_loop(messages)
        return 0
    except Exception as exc:
        print(f"ERROR: {type(exc).__name__}: {exc}")
        return 1
    finally:
        HOOKS.fire("session_end", {"noninteractive": True})


def main() -> None:
    parser = argparse.ArgumentParser(add_help=True)
    parser.add_argument("--prompt", help="Run one non-interactive task and exit")
    parser.add_argument("--agent", help="Run --prompt through a named subagent")
    parser.add_argument("--worker", action="store_true", help="Internal marker for team worker processes")
    args = parser.parse_args()

    if args.prompt:
        raise SystemExit(run_one_prompt(args.prompt, args.agent))

```

```
    print(f"Mini Claude Code s10 - workspace: {WORKSPACE}")
    print("Commands: /context /compact /clear /memory /skills /agents /sandbox /exit")
    HOOKS.fire("session_start", {"model": MODEL})
    messages: list[dict[str, Any]] = []
    while True:
        try:
            prompt = input("\nyou> ").strip()
        except (EOFError, KeyboardInterrupt):
            print()
            break
        if not prompt:
            continue
        if prompt in {"/exit", "/quit"}:
            break
        if prompt == "/context":
            print(CONTEXT.report(messages))
            continue
        if prompt == "/compact":
            HOOKS.fire("pre_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
            messages = CONTEXT.compact(messages, summarize_history)
            HOOKS.fire("post_compact", {"reason": "manual", "context": CONTEXT.report(messages)})
            print("[context] " + CONTEXT.report(messages))
            continue
        if prompt == "/clear":
            messages = []
            print("[context] cleared")
            continue
        if prompt == "/memory":
            print(MEMORY.load_auto_memory() or "(memory is empty)")
            continue
        if prompt == "/skills":
            print(SKILLS.catalog())
            continue
        if prompt == "/agents":
            print(AGENTS.catalog())
            continue
        if prompt == "/sandbox":
            print(SANDBOX.status())
            continue
        messages.append({"role": "user", "content": prompt})
        try:
            messages = agent_loop(messages)
        except Exception as exc:
            print(f"ERROR: {exc}")

    HOOKS.fire("session_end", {"message_count": len(messages)})


if __name__ == "__main__":
    main()
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/explorer.md (13 lines)

```
---
name: explorer
description: Read-only codebase exploration and evidence gathering.
tools:
  - read_file
  - glob
  - bash
max_turns: 10
---

You are a read-only exploration specialist.
Do not modify files. Trace code paths, gather evidence, and return a concise report with exact file paths.
Prefer search and focused reads over dumping large files.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/implementer.md (18 lines)

```
---
name: implementer
description: Implement a narrowly scoped change in an isolated worktree and verify it.
tools:
  - read_file
  - glob
  - write_file
  - edit_file
  - bash
```

```
0010    - use_skill
0011    - team_send
0012    - team_inbox
0013  max_turns: 18
0014  ---
0015
0016  You own the assigned task in an isolated Git worktree.
0017  Inspect before editing, keep the diff focused, run relevant tests, and report what changed.
0018  Do not merge branches. If another worker needs a discovery, send a concise message with team_send.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/agents/reviewer.md (12 lines)

```
0001  ---
0002  name: reviewer
0003  description: Review a proposed change for correctness, regressions, and missing tests.
0004  tools:
0005    - read_file
0006    - glob
0007    - bash
0008  max_turns: 10
0009  ---
0010
0011  Review independently. Inspect the diff and relevant tests. Return findings ordered by severity, with evidence.
0012  Do not edit files.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/audit_hook.py (12 lines)

```
0001  #!/usr/bin/env python3
0002  """Example hook: append compact JSON events to .mini_claude/audit.jsonl."""
0003  import json
0004  import sys
0005  from pathlib import Path
0006
0007  event = json.load(sys.stdin)
0008  path = Path('.mini_claude/audit.jsonl')
0009  path.parent.mkdir(parents=True, exist_ok=True)
0010  with path.open('a', encoding='utf-8') as fh:
0011      fh.write(json.dumps(event, ensure_ascii=False, default=str) + '\n')
0012  print(json.dumps({"decision": "allow"}))
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/hooks.json (8 lines)

```
0001  {
0002    "session_start": ["python .mini_claude/audit_hook.py"],
0003    "pre_tool_use": ["python .mini_claude/audit_hook.py"],
0004    "post_tool_use": ["python .mini_claude/audit_hook.py"],
0005    "pre_compact": ["python .mini_claude/audit_hook.py"],
0006    "post_compact": ["python .mini_claude/audit_hook.py"],
0007    "session_end": ["python .mini_claude/audit_hook.py"]
0008  }
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

.mini_claude/skills/verify-change/SKILL.md (12 lines)

```
0001  ---
0002  name: verify-change
0003  description: Verify a code change before declaring the task complete.
0004  ---
0005
0006  When implementation appears complete:
0007  1. inspect `git diff --stat` and `git diff`
0008  2. run the narrowest relevant test first
0009  3. run the broader regression suite if practical
0010  4. run lint/typecheck if the repository has them
0011  5. report exact commands and outcomes
0012  6. flag anything that was not verified
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

README.md (15 lines)

```
0001  # s10 Multi-Agent Runtime
0002
0003  Adds:
0004  - `run_team` tool
0005  - parallel worker processes
0006  - per-worker Git worktrees and branches
0007  - file-backed team mailbox (`team_send` / `team_inbox`)
0008  - non-interactive worker mode
```

```
- no automatic merge: changed worktrees are left for review

Example user request inside the REPL:

> Split this migration into three independent tasks. Use run_team with implementer workers and keep each worker on its own files.

The lead receives a JSON report with branch/worktree, diff stat, Git status, and worker output.
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

agents.py (69 lines)

```python
"""Custom subagent definitions for Mini Claude Code."""
from __future__ import annotations

from dataclasses import dataclass
from pathlib import Path
from typing import Any

import yaml


@dataclass(frozen=True)
class AgentDefinition:
    name: str
    description: str
    tools: list[str]
    model: str | None
    max_turns: int
    prompt: str
    metadata: dict[str, Any]


class AgentRegistry:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.root = self.workspace / ".mini_claude" / "agents"

    @staticmethod
    def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
        if not text.startswith("---\n"):
            return {}, text
        end = text.find("\n---\n", 4)
        if end == -1:
            return {}, text
        meta = yaml.safe_load(text[4:end]) or {}
        return (meta if isinstance(meta, dict) else {}), text[end + 5 :].strip()

    def discover(self) -> dict[str, AgentDefinition]:
        found: dict[str, AgentDefinition] = {}
        if not self.root.exists():
            return found
        for path in sorted(self.root.glob("*.md")):
            meta, body = self._split_frontmatter(path.read_text(encoding="utf-8", errors="replace"))
            name = str(meta.get("name") or path.stem)
            raw_tools = meta.get("tools") or ["read_file", "glob", "bash"]
            if isinstance(raw_tools, str):
                raw_tools = [x.strip() for x in raw_tools.split(",") if x.strip()]
            tools = [str(x) for x in raw_tools]
            found[name] = AgentDefinition(
                name=name,
                description=str(meta.get("description") or "No description provided"),
                tools=tools,
                model=str(meta["model"]) if meta.get("model") else None,
                max_turns=int(meta.get("max_turns", 12)),
                prompt=body,
                metadata=meta,
            )
        return found

    def catalog(self) -> str:
        agents = self.discover()
        if not agents:
            return "(no custom subagents installed)"
        return "\n".join(
            f"- {name}: {agent.description} [tools={','.join(agent.tools)}]"
            for name, agent in agents.items()
        )

    def get(self, name: str) -> AgentDefinition | None:
        return self.discover().get(name)
```

Mini Claude Code s01-s10 | 93

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

context_manager.py (59 lines)

```
"""A small context manager: estimate, inspect, and compact message history."""
from __future__ import annotations

import json
from dataclasses import dataclass
from typing import Any, Callable


@dataclass
class ContextStats:
    message_count: int
    approx_tokens: int
    chars: int


class ContextManager:
    def __init__(self, *, max_approx_tokens: int = 24000, keep_recent_messages: int = 8):
        self.max_approx_tokens = max_approx_tokens
        self.keep_recent_messages = keep_recent_messages

    @staticmethod
    def estimate(messages: list[dict[str, Any]]) -> ContextStats:
        raw = json.dumps(messages, ensure_ascii=False, default=str)
        # Deliberately simple heuristic. Production systems should use model-aware counting.
        return ContextStats(len(messages), max(1, len(raw) // 4), len(raw))

    def needs_compaction(self, messages: list[dict[str, Any]]) -> bool:
        return self.estimate(messages).approx_tokens >= self.max_approx_tokens

    def report(self, messages: list[dict[str, Any]]) -> str:
        stats = self.estimate(messages)
        pct = 100 * stats.approx_tokens / self.max_approx_tokens
        return (
            f"messages={stats.message_count}, approx_tokens={stats.approx_tokens:,}, "
            f"budget={self.max_approx_tokens:,} ({pct:.1f}%)"
        )

    def compact(
        self,
        messages: list[dict[str, Any]],
        summarizer: Callable[[list[dict[str, Any]]], str],
    ) -> list[dict[str, Any]]:
        if len(messages) <= self.keep_recent_messages + 2:
            return messages

        cut = len(messages) - self.keep_recent_messages
        old = messages[:cut]
        recent = messages[cut:]
        summary = summarizer(old)
        compacted = {
            "role": "user",
            "content": (
                "[COMPACTED_CONTEXT]\n"
                "The following is a structured summary of earlier conversation state. "
                "Treat it as historical context, not a new user request.\n\n"
                + summary
            ),
        }
        return [compacted, *recent]
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

hooks.py (82 lines)

```
"""Deterministic lifecycle hooks for Mini Claude Code."""
from __future__ import annotations

import json
import subprocess
from dataclasses import dataclass
from pathlib import Path
from typing import Any


@dataclass
class HookResult:
    decision: str = "allow"   # allow | deny | ask
```

```
    reason: str = ""
    updated_input: dict[str, Any] | None = None
    output: str = ""


class HookManager:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.config_path = self.workspace / ".mini_claude" / "hooks.json"

    def _config(self) -> dict[str, Any]:
        if not self.config_path.exists():
            return {}
        try:
            data = json.loads(self.config_path.read_text(encoding="utf-8"))
            return data if isinstance(data, dict) else {}
        except Exception as exc:
            print(f"[hook] invalid config: {exc}")
            return {}

    def fire(self, event: str, payload: dict[str, Any]) -> HookResult:
        commands = self._config().get(event, [])
        if isinstance(commands, str):
            commands = [commands]
        result = HookResult()
        for command in commands:
            if not isinstance(command, str) or not command.strip():
                continue
            envelope = {"event": event, "cwd": str(self.workspace), **payload}
            try:
                proc = subprocess.run(
                    command,
                    shell=True,
                    cwd=self.workspace,
                    input=json.dumps(envelope, ensure_ascii=False),
                    text=True,
                    capture_output=True,
                    timeout=15,
                )
            except subprocess.TimeoutExpired:
                return HookResult(decision="deny", reason=f"hook timed out: {command}")

            if proc.stderr.strip():
                print(f"[hook:{event}:stderr] {proc.stderr.strip()[:2000]}")
            if proc.returncode != 0:
                return HookResult(decision="deny", reason=f"hook failed ({proc.returncode}): {command}")

            stdout = proc.stdout.strip()
            if not stdout:
                continue
            try:
                data = json.loads(stdout)
            except json.JSONDecodeError:
                result.output += stdout + "\n"
                continue

            if isinstance(data, dict):
                decision = str(data.get("decision", "allow"))
                if decision in {"allow", "deny", "ask"}:
                    result.decision = decision
                if data.get("reason"):
                    result.reason = str(data["reason"])
                if isinstance(data.get("updated_input"), dict):
                    result.updated_input = data["updated_input"]
                if data.get("output"):
                    result.output += str(data["output"]) + "\n"
                if result.decision == "deny":
                    return result
        return result
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

memory.py (59 lines)

```
"""Persistent project instructions + agent-written memory for Mini Claude Code."""
from __future__ import annotations

from pathlib import Path
from typing import Iterable


```

Mini Claude Code s01-s10 | 95

```python
class MemoryStore:
    def __init__(self, workspace: Path):
        self.workspace = workspace.resolve()
        self.project_instructions = self.workspace / "CLAUDE.md"
        self.memory_dir = self.workspace / ".mini_claude" / "memory"
        self.memory_file = self.memory_dir / "MEMORY.md"

    def load_project_instructions(self) -> str:
        if not self.project_instructions.exists():
            return ""
        return self.project_instructions.read_text(encoding="utf-8", errors="replace")[:30000]

    def load_auto_memory(self) -> str:
        if not self.memory_file.exists():
            return ""
        # Keep startup memory intentionally bounded; detailed topic files can be read via file tools.
        text = self.memory_file.read_text(encoding="utf-8", errors="replace")
        return "\n".join(text.splitlines()[:200])[:25000]

    def remember(self, note: str) -> str:
        note = note.strip()
        if not note:
            return "ERROR: empty memory note"
        self.memory_dir.mkdir(parents=True, exist_ok=True)
        existing = self.memory_file.read_text(encoding="utf-8", errors="replace") if self.memory_file.exists() else "# Mini Claude Memory\n\n"
        normalized = f"- {note}"
        if normalized in existing:
            return "OK: note already present"
        with self.memory_file.open("a", encoding="utf-8") as fh:
            if not existing.endswith("\n"):
                fh.write("\n")
            fh.write(normalized + "\n")
        return "OK: persisted memory"

    def search(self, query: str, limit: int = 20) -> str:
        if not self.memory_file.exists():
            return "(memory is empty)"
        q = query.casefold().strip()
        lines = self.memory_file.read_text(encoding="utf-8", errors="replace").splitlines()
        hits: Iterable[str] = (line for line in lines if q in line.casefold()) if q else lines
        selected = list(hits)[:limit]
        return "\n".join(selected) if selected else "(no memory matches)"

    def startup_context(self) -> str:
        parts = []
        project = self.load_project_instructions().strip()
        memory = self.load_auto_memory().strip()
        if project:
            parts.append("<project_instructions>\n" + project + "\n</project_instructions>")
        if memory:
            parts.append("<auto_memory>\n" + memory + "\n</auto_memory>")
        return "\n\n".join(parts)
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

sandbox.py (102 lines)

```python
"""Execution boundary for shell tools.

`soft` mode is portability-oriented and is NOT a strong OS security boundary.
`bwrap` mode uses Bubblewrap on Linux when available and can isolate network + filesystem.
"""
from __future__ import annotations

import os
import shutil
import subprocess
import tempfile
from dataclasses import dataclass
from pathlib import Path


@dataclass(frozen=True)
class SandboxResult:
    exit_code: int
    stdout: str
    stderr: str
    mode: str


class SandboxRunner:
```

Mini Claude Code s01-s10 | 96

```
    def __init__(self, workspace: Path, *, mode: str = "auto", disable_network: bool = True):
        self.workspace = workspace.resolve()
        self.requested_mode = mode
        self.disable_network = disable_network
        self.bwrap = shutil.which("bwrap")

    @property
    def mode(self) -> str:
        if self.requested_mode == "auto":
            return "bwrap" if self.bwrap else "soft"
        if self.requested_mode == "bwrap" and not self.bwrap:
            raise RuntimeError("MINICLAUDE_SANDBOX=bwrap requested but `bwrap` is not installed")
        if self.requested_mode not in {"soft", "bwrap"}:
            raise ValueError("sandbox mode must be auto, soft, or bwrap")
        return self.requested_mode

    def sanitized_env(self) -> dict[str, str]:
        allowed = {"PATH", "LANG", "LC_ALL", "TERM", "SHELL", "USER"}
        env = {k: v for k, v in os.environ.items() if k in allowed}
        env["HOME"] = str(self.workspace / ".mini_claude" / "sandbox_home")
        env["MINICLAUDE_SANDBOX"] = self.mode
        Path(env["HOME"]).mkdir(parents=True, exist_ok=True)
        return env

    def run(self, command: str, *, timeout: int = 30) -> SandboxResult:
        mode = self.mode
        env = self.sanitized_env()
        if mode == "soft":
            proc = subprocess.run(
                ["bash", "-lc", command],
                cwd=self.workspace,
                env=env,
                text=True,
                capture_output=True,
                timeout=timeout,
            )
            return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)

        # Bubblewrap: workspace is writable; common system paths are read-only.
        # This is intentionally conservative and may need distro-specific adjustments.
        tmp = tempfile.mkdtemp(prefix="mini-claude-")
        argv = [
            self.bwrap or "bwrap",
            "--die-with-parent",
            "--new-session",
            "--proc", "/proc",
            "--dev", "/dev",
            "--tmpfs", "/tmp",
            "--bind", str(self.workspace), str(self.workspace),
            "--chdir", str(self.workspace),
        ]
        for system_path in ("/usr", "/bin", "/lib", "/lib64", "/etc"):
            if Path(system_path).exists():
                argv += ["--ro-bind", system_path, system_path]
        if self.disable_network:
            argv.append("--unshare-net")
        argv += ["--setenv", "HOME", str(self.workspace / ".mini_claude" / "sandbox_home")]
        argv += ["bash", "-lc", command]

        try:
            proc = subprocess.run(
                argv,
                cwd=self.workspace,
                env=env,
                text=True,
                capture_output=True,
                timeout=timeout,
            )
            return SandboxResult(proc.returncode, proc.stdout, proc.stderr, mode)
        finally:
            shutil.rmtree(tmp, ignore_errors=True)

    def status(self) -> str:
        return (
            f"requested={self.requested_mode}, active={self.mode}, "
            f"network={'disabled' if self.disable_network and self.mode == 'bwrap' else 'not strongly isolated'}, "
            f"workspace={self.workspace}"
        )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 97

skills.py (70 lines)

```
0001 """Progressive-disclosure skills for Mini Claude Code."""
0002 from __future__ import annotations
0003
0004 from dataclasses import dataclass
0005 from pathlib import Path
0006 from typing import Any
0007
0008 import yaml
0009
0010
0011 @dataclass(frozen=True)
0012 class Skill:
0013     name: str
0014     description: str
0015     path: Path
0016     body: str
0017     metadata: dict[str, Any]
0018
0019
0020 class SkillRegistry:
0021     def __init__(self, workspace: Path):
0022         self.workspace = workspace.resolve()
0023         self.roots = [
0024             self.workspace / ".mini_claude" / "skills",
0025             self.workspace / ".claude" / "skills",  # optional compatibility
0026         ]
0027
0028     @staticmethod
0029     def _split_frontmatter(text: str) -> tuple[dict[str, Any], str]:
0030         if not text.startswith("---\n"):
0031             return {}, text
0032         end = text.find("\n---\n", 4)
0033         if end == -1:
0034             return {}, text
0035         raw = text[4:end]
0036         body = text[end + 5 :]
0037         data = yaml.safe_load(raw) or {}
0038         if not isinstance(data, dict):
0039             data = {}
0040         return data, body
0041
0042     def discover(self) -> dict[str, Skill]:
0043         found: dict[str, Skill] = {}
0044         for root in self.roots:
0045             if not root.exists():
0046                 continue
0047             for skill_file in sorted(root.glob("*/SKILL.md")):
0048                 text = skill_file.read_text(encoding="utf-8", errors="replace")
0049                 meta, body = self._split_frontmatter(text)
0050                 name = str(meta.get("name") or skill_file.parent.name)
0051                 description = str(meta.get("description") or "No description provided")
0052                 found[name] = Skill(name, description, skill_file, body.strip(), meta)
0053         return found
0054
0055     def catalog(self) -> str:
0056         skills = self.discover()
0057         if not skills:
0058             return "(no skills installed)"
0059         return "\n".join(f"- {name}: {skill.description}" for name, skill in skills.items())
0060
0061     def load(self, name: str) -> str:
0062         skill = self.discover().get(name)
0063         if skill is None:
0064             return f"ERROR: unknown skill {name!r}. Available:\n{self.catalog()}"
0065         # A skill can reference support files by relative path; normal file tools can read them later.
0066         return (
0067             f"# Skill: {skill.name}\n"
0068             f"Source: {skill.path.relative_to(self.workspace)}\n\n"
0069             f"{skill.body[:30000]}"
0070         )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

team.py (260 lines)

```
0001 """Parallel multi-agent runtime with Git worktree isolation and a tiny mailbox."""
0002 from __future__ import annotations
0003
0004 import json
```

Mini Claude Code s01-s10 | 98

```
import os
import re
import shutil
import subprocess
import sys
from concurrent.futures import ThreadPoolExecutor, as_completed
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from typing import Any


@dataclass(frozen=True)
class WorkerSpec:
    name: str
    agent: str
    prompt: str


class TeamBus:
    """Very small file-backed mailbox shared by worker processes."""

    def __init__(self):
        raw = os.getenv("MINICLAUDE_TEAM_DIR")
        self.root = Path(raw).resolve() if raw else None
        self.me = os.getenv("MINICLAUDE_WORKER_NAME")
        try:
            self.members = json.loads(os.getenv("MINICLAUDE_TEAM_MEMBERS", "[]"))
        except json.JSONDecodeError:
            self.members = []

    @property
    def enabled(self) -> bool:
        return bool(self.root and self.me)

    def context(self) -> str:
        if not self.enabled:
            return ""
        return (
            f"You are teammate {self.me}. Team members: {', '.join(self.members)}. "
            "Use team_send for concise discoveries that unblock another teammate. "
            "Use team_inbox when you expect a reply."
        )

    def send(self, to: str, message: str) -> str:
        if not self.enabled or self.root is None or self.me is None:
            return "ERROR: not running inside a team"
        if to not in self.members:
            return f"ERROR: unknown teammate {to!r}"
        mailbox = self.root / "mailboxes" / f"{to}.jsonl"
        mailbox.parent.mkdir(parents=True, exist_ok=True)
        record = {
            "from": self.me,
            "to": to,
            "time": datetime.now(timezone.utc).isoformat(),
            "message": message,
        }
        with mailbox.open("a", encoding="utf-8") as fh:
            fh.write(json.dumps(record, ensure_ascii=False) + "\n")
        return f"OK: sent message to {to}"

    def inbox(self) -> str:
        if not self.enabled or self.root is None or self.me is None:
            return "ERROR: not running inside a team"
        mailbox = self.root / "mailboxes" / f"{self.me}.jsonl"
        if not mailbox.exists():
            return "(inbox empty)"
        lines = mailbox.read_text(encoding="utf-8", errors="replace").splitlines()
        return "\n".join(lines[-50:]) if lines else "(inbox empty)"


class WorktreeManager:
    def __init__(self, repo: Path):
        self.repo = repo.resolve()
        self.root = self.repo / ".mini_claude" / "worktrees"

    def _git(self, *args: str, check: bool = True) -> subprocess.CompletedProcess[str]:
        return subprocess.run(
            ["git", "-C", str(self.repo), *args],
            text=True,
```

Mini Claude Code s01-s10 | 99

```
            capture_output=True,
            check=check,
        )

    def is_git_repo(self) -> bool:
        proc = self._git("rev-parse", "--is-inside-work-tree", check=False)
        return proc.returncode == 0 and proc.stdout.strip() == "true"

    @staticmethod
    def slug(text: str) -> str:
        value = re.sub(r"[^a-zA-Z0-9._-]+", "-", text).strip("-.")
        return value[:60] or "worker"

    def create(self, team_name: str, worker_name: str) -> tuple[Path, str]:
        if not self.is_git_repo():
            raise RuntimeError("multi-agent worktree mode requires a Git repository")
        team_slug = self.slug(team_name)
        worker_slug = self.slug(worker_name)
        branch = f"mini/{team_slug}/{worker_slug}"
        path = (self.root / team_slug / worker_slug).resolve()
        path.parent.mkdir(parents=True, exist_ok=True)
        if path.exists():
            shutil.rmtree(path)
        proc = subprocess.run(
            ["git", "-C", str(self.repo), "worktree", "add", "-b", branch, str(path), "HEAD"],
            text=True,
            capture_output=True,
        )
        if proc.returncode != 0:
            raise RuntimeError(proc.stderr.strip() or proc.stdout.strip())
        return path, branch

    @staticmethod
    def status(path: Path) -> tuple[str, str]:
        status = subprocess.run(
            ["git", "-C", str(path), "status", "--short"],
            text=True,
            capture_output=True,
        ).stdout.strip()
        diff = subprocess.run(
            ["git", "-C", str(path), "diff", "--stat"],
            text=True,
            capture_output=True,
        ).stdout.strip()
        return status, diff

    def remove_if_clean(self, path: Path, branch: str) -> bool:
        status, _ = self.status(path)
        if status:
            return False
        subprocess.run(
            ["git", "-C", str(self.repo), "worktree", "remove", "--force", str(path)],
            text=True,
            capture_output=True,
        )
        subprocess.run(
            ["git", "-C", str(self.repo), "branch", "-D", branch],
            text=True,
            capture_output=True,
        )
        return True


class TeamRuntime:
    def __init__(self, workspace: Path, entry_script: Path):
        self.workspace = workspace.resolve()
        self.entry_script = entry_script.resolve()
        self.worktrees = WorktreeManager(self.workspace)

    def run(self, team_name: str, tasks: list[dict[str, Any]]) -> str:
        if not tasks:
            return "ERROR: tasks must not be empty"
        if len(tasks) > 8:
            return "ERROR: tutorial runtime limits a team to 8 workers"

        specs: list[WorkerSpec] = []
        names: set[str] = set()
        for i, item in enumerate(tasks, start=1):
            name = WorktreeManager.slug(str(item.get("name") or f"worker-{i}"))
            if name in names:
```

Mini Claude Code s01-s10 | 100

```
                return f"ERROR: duplicate worker name {name!r}"
            names.add(name)
            specs.append(
                WorkerSpec(
                    name=name,
                    agent=str(item.get("agent") or "implementer"),
                    prompt=str(item.get("prompt") or "").strip(),
                )
            )
        if any(not spec.prompt for spec in specs):
            return "ERROR: every worker needs a non-empty prompt"

        team_slug = WorktreeManager.slug(team_name)
        team_dir = (self.workspace / ".mini_claude" / "teams" / team_slug).resolve()
        team_dir.mkdir(parents=True, exist_ok=True)
        members = [s.name for s in specs]
        (team_dir / "team.json").write_text(
            json.dumps({"team": team_name, "members": members}, ensure_ascii=False, indent=2),
            encoding="utf-8",
        )

        assignments: list[tuple[WorkerSpec, Path, str]] = []
        try:
            for spec in specs:
                path, branch = self.worktrees.create(team_name, spec.name)
                assignments.append((spec, path, branch))
        except Exception as exc:
            return f"ERROR creating worktrees: {type(exc).__name__}: {exc}"

        def worker_job(spec: WorkerSpec, path: Path, branch: str) -> dict[str, Any]:
            env = os.environ.copy()
            env.update(
                {
                    "MINICLAUDE_NONINTERACTIVE": "1",
                    "MINICLAUDE_WORKER_ALLOW_EDITS": "1",
                    "MINICLAUDE_TEAM_DIR": str(team_dir),
                    "MINICLAUDE_WORKER_NAME": spec.name,
                    "MINICLAUDE_TEAM_MEMBERS": json.dumps(members),
                }
            )
            proc = subprocess.run(
                [
                    sys.executable,
                    str(self.entry_script),
                    "--worker",
                    "--agent",
                    spec.agent,
                    "--prompt",
                    spec.prompt,
                ],
                cwd=path,
                env=env,
                text=True,
                capture_output=True,
                timeout=900,
            )
            status, diff = self.worktrees.status(path)
            cleaned = False
            if proc.returncode == 0 and not status:
                cleaned = self.worktrees.remove_if_clean(path, branch)
            return {
                "name": spec.name,
                "agent": spec.agent,
                "branch": branch,
                "worktree": str(path),
                "exit_code": proc.returncode,
                "stdout": proc.stdout[-12000:],
                "stderr": proc.stderr[-4000:],
                "git_status": status,
                "diff_stat": diff,
                "cleaned_up": cleaned,
            }

        results: list[dict[str, Any]] = []
        with ThreadPoolExecutor(max_workers=len(assignments)) as pool:
            futures = {
                pool.submit(worker_job, spec, path, branch): spec.name
                for spec, path, branch in assignments
            }
            for future in as_completed(futures):
```

Mini Claude Code s01-s10 | 101

```
                name = futures[future]
                try:
                    results.append(future.result())
                except Exception as exc:
                    results.append({"name": name, "error": f"{type(exc).__name__}: {exc}"})

        results.sort(key=lambda x: str(x.get("name", "")))
        return json.dumps(
            {
                "team": team_name,
                "note": "Changed worktrees are intentionally left for review; this runtime never auto-merges branches.",
                "results": results,
            },
            ensure_ascii=False,
            indent=2,
        )
```

注：PDF 行号用于讨论；可运行无行号版本见随附源码 ZIP。

Mini Claude Code s01-s10 | 102

12. 从教程版走向可维护 Runtime：重构路线

s01-s10 故意强调“每层能看懂”，因此 code.py 逐步变长。真正维护时应在理解完成后再做一次结构化重构，而不是一开始就把所有抽象拆成十几个目录。

12.1 推荐最终包结构

```
mini_claude/
  app.py              # CLI / REPL / non-interactive entry
  llm.py              # Messages API adapter
  loop.py             # agent loop state machine
  tools/              # schemas + handlers
  policy/             # permissions
  context.py           # budget + compaction
  memory.py            # instructions + durable memory
  skills.py            # discovery + loading
  hooks.py             # lifecycle bus
  agents.py            # subagent registry/runtime
  sandbox.py           # process isolation adapters
  team.py              # worktrees + worker orchestration
  telemetry.py         # events / costs / traces
.claude or .mini_claude/
  skills/
  agents/
  settings / hooks / memory
```

12.2 生产化还缺什么

- 模型感知的精确 token counting、prompt caching、streaming 与 retry/backoff。
- 结构化事件/trace：每轮模型输入、工具调用、权限决定、Hook 决定、成本与延迟。
- 更严格的 Tool Schema validation、输出类型、并行 tool call 调度和幂等性。
- 强 OS sandbox/容器/VM、网络 egress proxy、secret broker、资源 quota。
- 非交互权限策略文件、组织级 policy、审计签名与策略版本。
- 可靠的 session persistence、JSONL/event sourcing、checkpoint/replay。
- MCP client/server 接入与 Tool Search；本教程故意把它留在 s10 之后作为下一阶段。
- Agent eval：任务成功率、工具选择准确率、permission friction、context efficiency、回归集。
- Team scheduler：依赖图、取消、超时、重试、任务窃取、消息可靠性和 merge/rebase 策略。

13. 调试手册：Agent 为什么“看起来很笨”

症状	根因	优先修法
工具老选错	Schema/description 含糊或相似工具重叠	缩窄工具边界；用 eval 验证选择率；不要只改 system prompt。
不断重复读文件	Context 中没有稳定的状态摘要	增加明确 plan/state；在适当节点 compact；Subagent 只回结论。
写完就说完成	没有可执行 verifier	把 test/lint/build/fixture/screenshot 作为 stopping condition。
频繁弹权限	allow policy 过窄或工具粒度太通用	把重复低风险操作变成明确 rule；高风险保留 ask。
CLAUDE.md 越写越长	把程序和临时知识都塞进启动 Context	事实放 CLAUDE.md，程序放 Skill，经验放 Memory，强制动作放 Hook。
Subagent 没省 Context	把主历史整段复制给 worker，或 worker 返回完整轨迹	只给 task + 必要 project context；返回 structured summary。
多 Agent 越多越慢	任务存在强依赖或 worker 争用同一文件	先做 dependency graph；只有真正可分割任务才并行。

Mini Claude Code s01-s10 | 103

症状	根因	优先修法
Sandbox 开了仍不放心	使用 soft 模式或网络/secret 仍共享	把 sandbox status 可见化；生产用 OS/VM containment + egress/secret policy。

14. 练习：把 Mini Claude Code 继续演进到 s11-s15

s11 Session Persistence

用 append-only JSONL 保存 user/assistant/tool events；实现 --resume 与 fork。

s12 MCP

实现 MCP client，动态注册 external tools；比较“schema 常驻”与 Tool Search。

s13 Verification Runtime

把 tests/lint/typecheck 变成 first-class verifier，支持失败反馈和 stopping policy。

s14 Eval Harness

为工具选择、权限、修复成功率建立离线任务集和可重复评测。

s15 Agent Team Scheduler

加入 DAG task board、worker heartbeat、可靠 mailbox、取消/超时、worktree merge policy。

15. 参考资料与公开机制映射

下列资料用于校准公开协议和产品概念。教程代码是原创教学实现；参考资料不意味着内部源码一一对应。

[R1] Claude Platform Docs - How tool use works
客户端工具的规范循环：tool_use -> Host execution -> tool_result -> continue。
https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works

[R2] Claude Platform Docs - Handle tool calls
tool_use block、tool_result、is_error 与 tool_use_id 的协议语义。
https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls

[R3] Claude Code Docs - How Claude Code works
Claude Code 的 Agentic loop、可访问资源、上下文与 compaction 概览。
https://code.claude.com/docs/en/how-claude-code-works

[R4] Claude Code Docs - How Claude remembers your project
CLAUDE.md、auto memory、加载范围和跨会话知识。
https://code.claude.com/docs/en/memory

[R5] Claude Code Docs - Extend Claude with skills
Skill 的按需加载、SKILL.md 与可复用程序。
https://code.claude.com/docs/en/slash-commands

[R6] Claude Code Docs - Hooks reference / guide
PreToolUse/PostToolUse、Session、Compact、Subagent、Worktree 等生命周期事件。
https://code.claude.com/docs/en/hooks

[R7] Claude Code Docs - Create custom subagents
独立 context、tool allowlist、max turns、isolation 等 Subagent 机制。
https://code.claude.com/docs/en/sub-agents

Mini Claude Code s01-s10 | 104

[R8] Claude Code Docs - Run parallel sessions with worktrees
使用 Git worktree 隔离并行会话和 Subagent 的文件修改。
https://code.claude.com/docs/en/worktrees

[R9] Claude Code Docs - Orchestrate teams of Claude Code sessions
Agent Teams 的独立 context、共享任务和消息协调；当前文档仍标注 experimental。
https://code.claude.com/docs/en/agent-teams

[R10] Claude Code Docs - Extend Claude Code / Tools reference
CLAUDE.md、Skills、Subagents、Hooks、MCP/Tools 在 Agent loop 中的扩展位置。
https://code.claude.com/docs/en/features-overview

16. 最终心智模型

一句话
真正的 Coding Agent 不是“LLM + 一个 Prompt”，而是 Model x Context x Tools x Policy x Feedback x Isolation x Lifecycle。
s01-s10 的目的，就是让这些乘数项一层层显形。

- Agent Loop 决定“能否继续行动”。
- Tools 决定“能做什么”。
- Permissions 决定“这次能不能做”。
- Context 决定“当前能想起什么”。
- Memory 决定“跨会话保留什么”。
- Skills 决定“程序性经验如何按需注入”。
- Hooks 决定“哪些动作必须确定发生”。
- Subagents 决定“哪些认知工作应隔离”。
- Sandbox 决定“最坏情况下能影响多大范围”。
- Multi-Agent Runtime 决定“如何把多个独立 Agent 变成一个可协调的软件工程系统”。

Mini Claude Code s01-s10 | 105